

Imperial College London

MENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Towards Rebound-Focused Reinforcement Learning: Solving Deadlocks in Traffic Intersections

Author:
Tiberiu-Andrei Georgescu

Supervisors:
Dr. Dario Paccagnan
Panagiotis Angeloudis

August 18, 2026

Abstract

The deployment of fully autonomous vehicles (AVs) in traffic is projected to occur within the next decade, and benefits like smaller congestion rates, less pollution, less time spent in traffic and safer traffic are expected to follow soon afterwards. For that to happen, the behaviour of the autonomous vehicles, as a collective, needs to surpass human drivers in terms of cooperation, coordination and damage mitigation, so that all traffic participants can feel safe.

This work proposes deadlock resolution in intersections occupied by homogenous AVs as an example scenario which, albeit improbable, combines features that the current State-of-the-Art in autonomous driving can be expected to have difficulties in resolving it safely. As a baseline for the problem, variations of multi-agent reinforcement learning algorithms have been trained to reach a resolution. The mixed results showcase the non-triviality of the problem.

This scenario highlighted the difficulty of preparing agents trained through Deep RL to be able to recover from a failed state, while concurrently preventing the occurrence of critical failures. This particular subset of problems, called *rebound*, is suggested as a potentially important area to shift the general focus of safe RL research towards.

In the pursuit of the aforementioned goals, a symbolic formalisation of the deadlock condition for continuous multi-agent systems is proposed, with relaxations suggested for the application of the condition in more practical situations, potentially outside the scope of autonomous driving.

Acknowledgements

This work is a cross-departmental project, supervised by Dr Dario Paccagnan, Lecturer in the Department of Computing, and by Dr Panagiotis Angeloudis, Reader in the Department of Civil and Environmental Engineering. Their contributions and support over the course of the year were instrumental for the development of this dissertation to its current and final state.

I would like to acknowledge Dr Francesco Belardinelli, Senior Lecturer in the Department of Computing, for his suggestions in the symbolic reasoning aspect of this dissertation, as well as his recommendations for literature review.

I would like to acknowledge the contributions of the Transport Systems and Logistics research team, whose members provided me with significant support on the research into relevant work, as well as the allocations of machines for the empirical aspect of this dissertation. I would like to emphasise the contributions of Dr Felix Feng, research associate, whose aid was significant in my initial introduction with the laboratory and research team. I would like to acknowledge Eduardo Candela Garza, Luis Marques and Leandro Parada, with whom I was able to publish work which influenced the outcome of this current dissertation; I would like to acknowledge Jules Dehon, whose collaboration helped in covering a higher amount of ground than it would have been possible by working alone on each of our dissertations, in terms of the literature and simulators we were able to review together.

Finally, I would like to acknowledge the contributions of my family, friends, and my peers in the Department of Computing and Department of Civil Engineering, for our discussions have helped in the discovery of new grounds related to the current work.

One last acknowledgement to everyone who was involved in the proof reading for the drafts of this report.

Contents

1	Introduction	4
1.1	Cooperative Autonomous Vehicles	4
1.2	Rebound from Failure	6
1.3	Resolution of Deadlocks in Traffic	8
1.4	Contributions	8
2	Background	10
2.1	Multi-Processing	10
2.1.1	Deadlocks	10
2.2	Deep Reinforcement Learning	12
2.2.1	Markov Decision Processes	12
2.2.2	Value-based Reinforcement Learning	13
2.2.3	Training Stabilisation	15
2.3	Multi-Agent Reinforcement Learning	17
2.3.1	Partially Observable Markovian Decision Process	17
2.3.2	Centralised Training Decentralised Execution	18
2.3.3	CTDE Extended	20
2.4	Learning Mechanisms	21
2.4.1	Recurrent Neural Networks	22
2.4.2	Attention	22
2.5	Related Work	24
2.5.1	Robustness in Reinforcement Learning	24
2.5.2	Deadlocks in Multi-Agent Systems	24
2.5.3	Deep Reinforcement Learning for Autonomous Driving	26
2.6	Background Key Takeaways	26
3	Methodology	28
3.1	Development Setup	28
3.1.1	Simulator Survey	28
3.1.2	Reinforcement Learning Library	29
3.1.3	Visualisation Tools and Data Collection	29
3.2	Rebound from Failure	30
3.2.1	The Mid-point between Robustness and Damage Mitigation	30
3.2.2	Definitions	31
3.3	Deadlocks in MAS	33
3.3.1	Softening Deadlocks for Continuous MAS	33
3.3.2	Deadlocks in Traffic	35
3.4	Simulated Environments	38
3.4.1	Deadlocked Starting State	38
3.4.2	Internal Representation	39
3.4.3	Vehicle Dynamics	40

3.5	Algorithms Implemented	41
3.5.1	DQN	41
3.5.2	MADDPG	42
4	Evaluation	45
4.1	Experimental Setup	45
4.1.1	Environments	45
4.1.2	Algorithms	46
4.2	Evaluation Results	46
4.2.1	Discussion	47
4.3	Algorithm Training Performance	47
4.3.1	Discussion	48
4.4	Key Takeaways	48
4.5	Challenges and Future Work	49
5	Ethical Issues	51
5.1	Protection of Personal Data	51
5.2	Misuse	51
5.2.1	Privacy	51
5.2.2	Safety and Relaxations over Rebound Condition	52
5.3	Workplace Disruption	53
6	Future Work	54
6.1	Rebound-Focused Reinforcement Learning	54
6.2	Deadlocks in MAS	54
6.3	Realistic Simulations & Sim2Real Transfer	54
6.4	Communication, Fairness & Scalability	55
6.5	Human in the Loop	56
6.6	General Cooperation in Traffic	56
6.7	Validation of More MARL Architectures	57
7	Conclusion	58
A	Algorithms in Background	61
A.1	Deep Q-Learning	61
A.2	Deep Deterministic Policy Gradient	62
A.3	Multi-Agent Deep Deterministic Policy Gradient (Actor-Critic)	63
A.4	Multi-Agent Attention Critic	64
B	Hyperparameters	65
B.1	Reward	65
B.1.1	Equation	65
B.1.2	Hyperparameter table	65
B.2	Training Networks	66
B.2.1	Hyperparameter Table	66

Chapter 1

Introduction

1.1 Cooperative Autonomous Vehicles

Autonomous driving has had a series of highly significant technical developments over the past decade [1], from improved perception capabilities [2], to more robust path planning and lane following, to the introduction of coordination through communication, in the form of Vehicle-to-Vehicle (V2V) and Vehicle-to-Everything (V2X) infrastructures [3]. These developments are already tested and deployed with partial autonomy on the road by manufacturers like Tesla and Waymo [4].

Having cooperative autonomous driven vehicles as a more pervasive part of world's traffic infrastructure is expected to have many benefits to society overall. Some of these benefits, mentioned in [5] as well, are:

1. Less Congestion, by proactively planning routes smartly through cooperation with other vehicles. Through game-theoretical models, an optimal traffic flow can be achieved by re-routing some agents on less direct routes, in order to avoid crowding common intersections and main traffic arteries¹. This directly leads to more traffic efficiency, and less time spent commuting for all individuals.
2. Less accidents, by preemptively warning the neighbouring vehicles before one vehicle takes an action, through intention sharing [6]. Human-driven vehicles are limited by the amount of information that can be sent over through light signals, and by the unreliability of human split-second reactions and decisions.
3. Less pollution, through congestion avoidance and through the introduction of autonomous on-demand fleets [7]. The former ensures less time is spent with a running engine for each individual vehicle, leading to less energy/fuel spent and a smaller carbon footprint. The latter is an option for households which rely on the transportation efficiency of a car, but do not need the ownership and wish to avoid the parking and maintenance costs. Through multiple households sharing a fleet of vehicles on-demand, the number of vehicles in traffic decreases, directly affecting congestion rates, and indirectly pollution rates.

Given the aforementioned motivation, and the rate of development of autonomous vehicle (AV) technology, it is forecast that fully autonomous vehicles will be deployed by the end of 2030, with the expected benefits becoming observable around 2050 [8].

¹<https://www.nytimes.com/1990/12/25/health/what-if-they-closed-42d-street-and-nobody-noticed.html>

However, with the current speed of development in [AV](#) technology, and with the high degree of reliance on Deep Learning ([DL](#)) approaches for much of the perception and planning capabilities of the vehicles, there comes the natural question of how *trustworthy* will these vehicles be once they become ubiquitous in real life traffic scenarios [\[9\]](#), particularly when met with unexpected circumstances.



Figure 1.1: Highly Deadlocked Traffic

A particularly problematic scenario to prepare for is the unsignalised intersection which, albeit the result of damaged signs, faulty signals or poor urban planning, they still occasionally occur in traffic, and require a high degree of skillful coordination from the participants to traverse. Even without highly expressive communication capabilities, human drivers are still generally capable of traversing such intersections safely, at the cost of a significant amount of throughput. However, in the case of high traffic flow, small mistakes are possible, and could very easily lead to an extremely congested intersection, to the point where a deadlock can occur, as seen in figure [1.1](#), making it significantly more difficult for traffic participants to rebound to a normal flow, as complex coordinated backwards maneuvers might be needed.

The aforementioned scenario poses a great problem for autonomous vehicles. Current iterations of [AVs](#) are not tested with this type of scenario in mind. Although autonomous agents should be able to traverse congested traffic, there is little evidence to suggest this form of coordinated maneuver is considered and trained for. On the same note, in the exceptional case where all agents are autonomous and find themselves in similar positions, with opposing destinations, unless the autonomous agents exhibit complex communication behaviour (i.e. reaching a consensus on the ordering of traversal), behaviour inference might not be enough and the agents will find themselves in a real conceptual deadlock, all of them waiting on each other to traverse the intersection. The conceptual deadlock, at least, can be avoided by human drivers in practice, due to the difference between each driver's temper and level of aggression. This usually leads to a natural resolution of the deadlock, potentially at the cost of fairness.

On the other hand, the same scenario also presents an opportunity for coordinating

cooperative AVs. State of the art intersection control algorithms propose various techniques for intersection traversal, from grid reservation to coordinated motion planning [10]. With these techniques, it is possible for the intersection’s spatial resources to be better allocated, leading to higher throughput on all incoming lanes and smaller rates of congestions. What is still explored to a smaller degree is the idea of faults in this coordination, and how the behaviour of AVs needs to change to prevent further accidents, as well as on how the agents could return from such a faulty state.

1.2 Rebound from Failure

Over the last 10 years, reinforcement learning methods have had a tremendous success in literature, from being generally good at playing Atari games [11], to surpassing the most skilled human players in strategy games like Chess [12], Go [13] and recently, DOTA 2 [14]. All developments over the years showcase reinforcement learning as a powerful tool, capable of high degrees of optimisation on the task being given, able to perform both cooperatively and competitively [15]. In short, given a clear goal and enough time, a RL algorithm should be able to outperform any human or non-learning system.

However, the usual optimisation methods may sometimes overlook other aspects of a particular problem, like safety or explainability. Even though reinforcement learning can showcase significantly better performance than many of the current systems in place, the fact that the behaviour of deep learning agents being trained cannot be perfectly understood by humans, makes people and governments reticent about deploying systems trained through training optimisation in places where system malfunctions could prove catastrophic [16, 17]. The problem of narrow-sighted optimisation is currently tackled by Bi-Level Optimisation [18], which constrains optimisation objectives by multiple conditions or optimisation criteria, whereas the general topic of safety is tackled by the safe reinforcement learning branch [19, 20], which works on modelling the exploration, exploitation and problem goals to provide safety guarantees that can be followed throughout training and execution.

Overall, the literature is highly focused on goal optimisation and danger avoidance, to the point where either the agent becomes too greedily focused on the improvement of its own policy, or too risk adverse, to the detriment of both itself and other agents sharing its environment, human or otherwise. For instance, reports [21] show that a significant majority of traffic accidents which include autonomous vehicles are rear-ended or rear-side crashes caused by a human driver, which showcase a proclivity from the AV to be highly risk adverse, and either drive at very slow speed, or break abruptly.

On a similar note, robustness has been used as a criteria of measuring safety, and is defined as the tolerance to perturbations of the system. The way in which this problem is usually trained is by introducing Neuron Coverage [22] or other types of noise in the network or training environment [22, 23], to make the agent behave more consistently to its observations in case of possible deviations, introduced either accidentally or by an adversary [24]. Unfortunately, due to the hyper-focus on optimisation, RL methods, by specialising in avoiding scenarios which could lead to critical failure, they raise the probability of reaching critical failure in cases where it can still be avoidable.

Please refer to figures 1.2 and 1.3 for an example and a theory. Let us take the example of storing a plate on a shelf, where partial failure occurs if the plate is free-falling, and critical failure occurs if the plate breaks. Intuitively, the expected chance of failure should

be linear, and directly proportional to the degree of risk in the activity. So the more oddly-shaped and slippery the plate, and the more turbulent the environment, the higher the possibility of dropping the plate and, consequently, critically failing the task. However, when it comes to humans, if they become aware that a failure occurred, the instincts of survival and self-preservation tend to provide the willpower and ingenuity to prevent the potentially catastrophic failures which can follow, allowing the recovery to a normal state later in the episode. On the same task though, a highly optimised RL agent will most likely train in a risk adverse way, by avoiding the first failure in the series at all costs. This strategy will help the agent to avoid partial and critical failures in a large majority of the time, at the expense of having no idea what to do to mitigate damages if that point is ever reached, due to under-training on that particular situation. For example, in the plate scenario, if the human was to drop the plate, it would do everything in its power to either catch the plate again from the air (rebound), or to dampen the effect of the fall to the point where the plate does not crash (damage mitigation). A robotic arm agent, on the other hand, would only try to grab the plate tighter and secure it better, making it more difficult for the plate to drop. However, if the plate was ever to drop, the agent would be under-prepared, and not do anything. This particular problem was highlighted by the DARPA competitions [25], where many of the humanoid robots unable to finish the tasks failed because they fell on their backs and were unable to get back up.

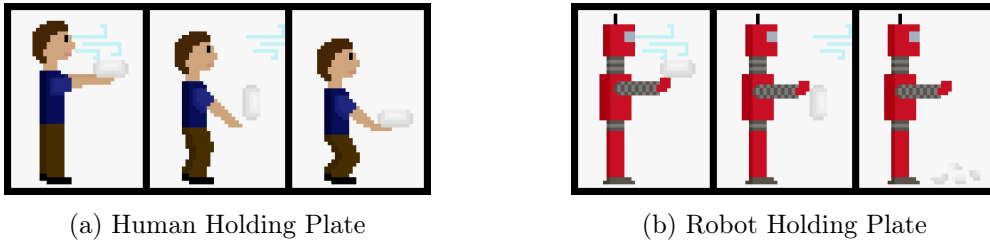


Figure 1.2: Holding Plates: Rebound from Dropping

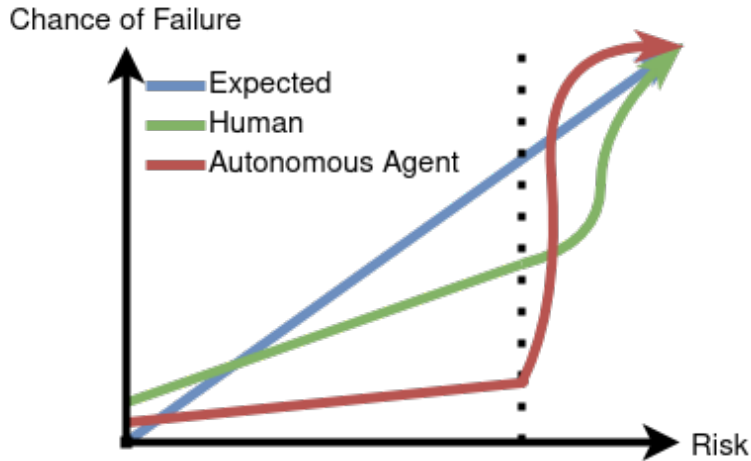


Figure 1.3: Risk to Chance of Failure Diagram

This work proposes a shift in perspective in the future of reinforcement learning, when training for robustness and safety. Instead of training for risk-aversion, which can lead to non-robustness in edge cases and less performance in general, the focus should be on being able to **rebound** from high risk situations. Rebound, by the definition of this thesis, is the set of actions taken with the purpose of recovering from failure-induced states to normal state, with the primary purpose of avoiding critical failures, from which it is impossible

to return. The proposal is inspired from human behaviour which, albeit unreliable and reckless, especially during developmental stages of life [26], it emphasises a proclivity for self-preservation, resourcefulness and empathy, making catastrophic failures more unlikely in high risk situations [27].

1.3 Resolution of Deadlocks in Traffic

Given the aforementioned subjects of interests, and their common limitations, this thesis proposes rebound from failure for the pursuit of safety in environments with autonomous agents deployed. This initial work will focus on the resolution of deadlocks in continuous multi-agent environments, particularly on traffic intersections populated with homogenous autonomous agents.

A deadlocked state for multiple AVs, for the scope of the project, is a cyclical state where multiple traffic agents are unable to take actions which get them closer to their destination. There are two main environments considered in the project: a simple cross intersection, and a lane-separated four way junction. The agents will be placed a priori in a state where it is impossible to move towards their goal without causing an accident, requiring them to coordinate and move backwards before cooperatively choosing which agent should move first. Such a deadlock can be caused by an accident, a path planning error, uncoordinated traffic participants, or other unpredictable system failure.

In AV and AGV literature, prevention and resolution of deadlocks is not an uncommon subject [28], and is generally solved through the modelling of specific agent behaviour, with higher focus on prevention rather than resolution. The literature primarily takes into account deadlocks in terms of trajectory planning, and the environment is simplified through layers of abstractions that discretise the state or action space. This process increases the potential for prevention, which allows the design of optimal deterministic policies. Furthermore, the simulated, or highly controlled environments in which AGVs are usually deployed (i.e. warehouse management) differs significantly from the highly dynamic traffic environments which current urban layouts provide, making failures occurring in the former less likely, and less costly.

For the continuous traffic environment, the definition of the deadlock had to be reimagined, with relaxations to enhance the practicality of the algorithms meant to detect their occurrence. This definition was designed with generality in mind, in the hopes of making it simple to transfer from one domain to another, particularly for continuous multi-agent systems (MAS).

The method of choice to tackle the problem of deadlocks in continuous traffic was deep, multi-agent reinforcement learning. Variations of two multi-agent reinforcement learning algorithms have been implemented, and the reward function has been modelled to prioritise the safe resolution of the deadlock, with incentive to pursue optimal throughput and ordering. All agents in the environment are autonomous and homogenous, so they share the same policy function. To discover emergent behaviour, provided the agents avoid crashing at all cost, traffic laws are relaxed during training and execution, in order to observe unique emergent behaviour with potential for faster and safer resolution.

1.4 Contributions

The main contributions of this dissertation are as follows:

- A proposal for the pursuit of *rebound* as a target for safety in reinforcement learning. This proposal is motivated by a brief review of the [RL](#) literature, highlighting limitations in the current ability of autonomous agents to recover from failed states, while avoiding critical failures. The proposed area of research is defined, and put in context with current state of the art.
- The proposal of deadlock resolution in traffic as a practical rebound-focused problem in the literature. This is motivated by a brief literature review, showcasing deadlock resolution as an underexplored area in traffic behaviour and planning for autonomous vehicles, as well as support of its importance for ensuring the safety of cooperative autonomous vehicles when deployed in real life.
- The formalisation of the conditions of a deadlock for Multi-Agent Systems with continuous dynamics, as well as further extensions which relax the condition of the deadlock’s occurrence, allowing the usage of the definition in more practical situations, like open space environments.
- Two customisable environments part of the Highway-Env repository [\[29\]](#) of OpenAI Gym compatible environments [\[30\]](#), for the development of strategies to resolve deadlocks in traffic intersections.
- The implementation and evaluation of multi-agent reinforcement learning policies trained on the given deadlocked environments, to serve as baselines for the research to be done on traffic deadlock resolution in future work.

In the pursuit of the aforementioned developments, contributions were made to two major open-source projects: *HighwayEnv* [\[29\]](#), a simulator for traffic scenarios and urban mobility, and *Rl-Agents* [\[31\]](#), a repository for reinforcement learning algorithms. A significant portion of the code contributions were upstreamed before the publication of this work, with the rest being in the process of being upstreamed, temporarily being maintained in personal forks of the projects ²³

The rest of this dissertation is structured as follows. In the background chapter, the fields influential to the scope and direction of this project is laid out, with detailed description of the theory used, as well as the important related work. In the methodology chapter, the majority of the contributions is explained. In the evaluation chapter, the experimental setup is explained, and experimental results are highlighted and commented on. In the ethical issues chapter, potential ethical considerations and methods of addressing them are discussed. In future work, a series of recommended and speculative directions for the continuation of this work are motivated. Finally, the conclusion summarises the contributions, as well as the prospects of the future work.

²<https://github.com/TibiGG/highway-env>

³<https://github.com/TibiGG/rl-agents>.

Chapter 2

Background

In this chapter presents the background theory necessary for the project to be undertaken. Furthermore, the current state of the art adaptations and improvements on the theory are detailed and referenced.

2.1 Multi-Processing

The usage of multiple parallel processes to solve problems has led to fast, scalable, highly efficient and robust systems [32, 33]. The development of synchronisation models to benefit from the availability of highly parallelised hardware, in scheduling and algorithm theory, is called multi-threaded programs, whereas in the field of distributed systems and artificial intelligence, this field is followed under the name of Multi-Agent Systems (MAS). These concurrent fields share a significant part of the SotA, as many of the developments in either field can be applied to the other under various layers of abstraction. This common work includes research into the detection, prevention and resolution of race conditions [34] and deadlocks [35].

2.1.1 Deadlocks

Deadlocks in scheduling theory occur when multiple threads or processes attempt to take a hold of the multiple shared resources at the same time. The threads simultaneously monopolise some resources and request access to others before they release the ones they currently own. If the sequence of threads which hold resources and request new ones ends up to be a cycle, then a deadlock occurred and the program cannot make any more progress. A deadlock in a MAS can be defined the same way, only with a potentially more abstract concept of resources, depending on the definition of the environment where the agents act in. In this section, "process" will be used to refer to either an agent in a MAS, or a core process/thread in a generic computer system, with the distinction made explicit when relevant.

There are 4 main conditions [35] necessary for a deadlock to occur in any type of system:

1. **Mutual Exclusion**, when the ownership of resources cannot be shared by two or more processes;
2. **Wait For**, when processes wait to gain the ownership of requested resources, if said resources are already owned by another process;
3. **No Preemption**, when it is not possible for resources to be removed by force from a process currently owning them;

4. **Circular Wait**, when a cycle of processes waiting on resources can be generated by the dynamics of the program which scheduled them.

A deadlock cannot occur unless all these conditions are satisfied by the program. The example in figure 2.1 represents a simple deadlock scenario. $P1$ and $P2$ represent processes in a program, $R1$ and $R2$ represent resources, arrows pointing from a process to a resource represents the *request for acquisition*, and arrows pointing from a resource to a process represent *ownership* or the process over that resource. Since the scenario exhibits all aforementioned conditions, a deadlock occurred and the program cannot make any more progress.

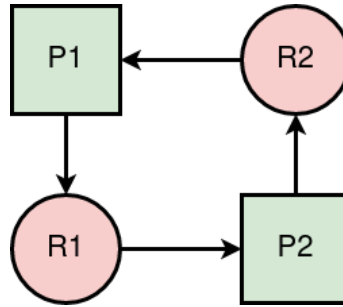


Figure 2.1: Simple Deadlock Example

Detection

The detection of a deadlock [36] can occur if, during the runtime of the algorithm, there exist a set of routines in place that allow for the discovery of a deadlock after its occurrence. The mechanism that allows the discovery of the deadlock can provide information on what processes got deadlocked, as well as what resources they are fighting over ownership for.

A standard way of detection is based on graph theory, where the mechanism is tasked with storing a graph representation of the requests and ownership of resources by the processes in the running task. This graph can be checked either during each modification, at set time intervals, or on certain triggers which the running task can provide. If at any point in time this graph contains a cycle like the one in figure 2.1, it means that a deadlock occurred and the task manager now needs to decide on how to resolve the problem. At the moment of a deadlock detection, the primary options the task manager can take are crashing, restarting, or recovering the program from the deadlock.

Prevention vs Recovery

Deadlock prevention occurs when tasks are planned in such a way as to avoid the 4 conditions of a deadlock's existence or, when not possible, to develop a strategy which ensures that the states which can potentially lead to a deadlock are avoided. Deadlock recovery can be done after the occurrence of a deadlock in a system, and works by taking actions which return from the deadlock to a normal running state.

Both strategies are needed only when, by default, a program allows the occurrence of a deadlock. Through deadlock prevention, the program is restructured in a way that restricts deadlocks from occurring. This strategy can have a significant overhead on certain components during the runtime of the program, leading to a worse overall performance compared to the default version, assuming that no deadlock would take place. The alternative, deadlock resolution, would add an overhead on the runtime of the program only if a

deadlock occurs. However, it is highly likely for the occurrence of the deadlocks to yield a significantly worse performance compared to a program with deadlock prevention. On the same note, the deadlock resolution strategy can be highly resource intensive. Therefore, if it is needed to run too frequently, it loses its practicality compared to prevention.

Therefore, a hybrid approach is taken in general, where prevention is used for elements of the program which have a high probability of leading to a deadlock, and resolution as a backup for unplanned or infrequent edge cases.

Livelock

The Livelock is a variation of a deadlock which occurs when multiple threads maintain a running state while repeatedly attempting to acquire one or more resources which other threads own. Usually, this variation happens when the design of deadlock prevention or resolution strategies is faulty, so it can occur even when one or multiple of the conditions necessary for a deadlock is missing. A baseline behaviour of a set of agents which find themselves in a livelock follows this logic:

1. Agent locks resource 1;
2. Agent attempts to lock resource 2;
3. Resource 2 is locked. Release both resources, wait n seconds and go back to 1.

This strategy is a naïve way of introducing a degree of preemption in the program. It ensures that agents do not wait for the acquisition of more resources while holding greedily onto others. However, unless the ordering for locking the resources is the same, or time n is random for each thread, it is possible for the loop to never end (or to finish in a very long time), resulting in a livelock.

2.2 Deep Reinforcement Learning

The solving of decision making problems in goal-directed environments has been saturated in the literature by Reinforcement Learning (RL) approaches. Compared to supervised learning approaches, RL does not require any form of dataset to begin training and, compared to evolutionary algorithms, on top of the need to optimise the output policy, there is a focus on the prediction of values from the reached states.

The introduction of Neural Networks (NNs) to Reinforcement Learning contributed to the fast evolution of the state of the art Deep Reinforcement Learning (DRL) algorithms, with developments like Mnih et. al.’s Atari Game learners [37, 11] showcasing its potential, and Lillicrap et. al.’s Deep Deterministic Policy Gradient (DDPG) algorithm [38] extending the abilities of RL into continuous action spaces.

2.2.1 Markov Decision Processes

The Markovian Decision Process (MDP) is the method of choice for the majority of reinforcement learning approaches to base the design of their environment on. It is a mathematical structure with convergence properties in dynamic programming scenarios, proven by Richard Bellman [39].

In a single-agent RL scenario, an MDP can be defined as a tuple $\langle S, A, P, R, \gamma \rangle$, with a state space S , action space A , dynamics function with probability $P \in \mathcal{M}(S)^{S \times A}$, with $\mathcal{M}(\mathcal{X})$ being the probability measures over the set of $\mathcal{X}$, reward function $R \in \mathbb{R}^{S \times A}$, and

cumulative-reward discount term γ .

Intuitively, during training and execution, there will be an Agent within an Environment. Assuming discrete time steps, at time t , the agent takes an action A_t , which is applied to the Environment in state S_t . The dynamics of the Environment and the reward function will return, based on S_t and A_t , a new state S_{t+1} and a reward R_{t+1} . Then, the state of the environment is updated, and the reward is used during training to optimise the agent's policy. A diagram of the process can be seen in figure 2.2.

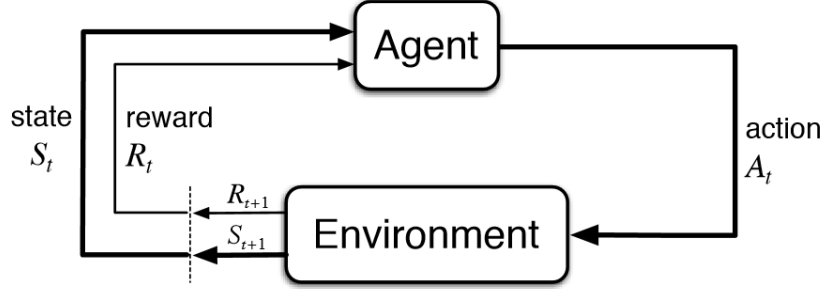


Figure 2.2: Agent-Environment interaction of an MDP [40]

2.2.2 Value-based Reinforcement Learning

A common method of solving an MDP environment with RL is through value-based optimisation. Assuming discrete time steps, through iterative experimentation in the environment $\mathcal{E}$, the agent attempts to learn a policy $\pi \in \mathcal{M}(A)^S$ based on approximating the optimal action-value function $Q^*(s, a)$ of $\mathcal{E}$. The policy is value-based, because it chooses its actions greedily, based on the estimation of the discounted sum of rewards $R(s, a)$ at each time step. Formally, the approximated value and action-value functions $V^\pi(s)$ and $Q^\pi(s, a)$, are defined as:

$$V^\pi(s) \stackrel{def}{=} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \middle| s_0 = s, a_t \sim \pi(a_t | s_t), s_{t+1} \sim P(s_{t+1} | s_t, a_t) \right]$$

$$Q^\pi(s, a) \stackrel{def}{=} R(s, a) + \gamma \mathbb{E}_{s' \sim P(s' | s, a)} V^\pi(s')$$

And the optimal action-value function $Q^*(s, a)$ is defined to satisfy the Bellman Optimality Equation:

$$Q^*(s, a) \stackrel{def}{=} \mathbb{E}_{s' \sim P(s' | s, a)} \max_{a' \in A} [R(s, a) + \gamma Q^*(s', a')]$$

On-policy vs Off-policy Reinforcement Learning

In the family of model-free reinforcement learning algorithms, there is a separation in terms of the chosen way to generate the behaviour policy, based on the design philosophy used to balance exploration and exploitation. These are on-policy and off-policy algorithms.

On-policy RL algorithms are optimised by direct experimentation with the behaviour policy in the environment. Off-policy RL algorithms take the approach of doing experimentation with one or multiple training policies in the given environment, then returning

a different behaviour policy, engineered using the experience of the training policies. There are variations of both on-policy and off-policy algorithms, but most have a baseline architecture they all follow. For on-policy, they are split in SARSA, Policy Optimisation and Actor-Critic-based algorithms, and off-policy algorithms are split in Time-Difference (TD) and Policy Gradient-based algorithms.

The benefits of choosing one over the other are somewhat balanced, and can depend on the scenario in which they are introduced. On-policy [RL](#) algorithms benefit from the fact that their behaviour policy is the same as the policy being used to train and explore, making training faster and more optimal for paths which are bound to reach good results. The downside could be that it is possible for such a policy to reach a local optimum and stop exploring. The off-policy method ensures that the agent maintains a reasonable degree of exploration during its training, as the behaviour policy being learned is not used during training, but only learns from the experience of the policy used during training. Furthermore, off-policy methods can benefit highly from parallelisation of training, since multiple exploration policies can be deployed on different machines, and the shared experiences can then be used to train the behaviour policy. This results in more general policies, at the cost of reduced precision.

Deep Q Network

In the family of TD learning off-policy algorithms, Q-learning [\[41\]](#) was one of the earliest breakthroughs [\[40\]](#). It proposes learning the action-value function $Q^\pi(s, a)$ to directly approximate the optimal $Q^*(s, a)$ function. This method simplified the convergence conditions, during indefinite training, to only require that all state-action value pairs are continuously updated. Formally, the update function to approximate Q^π is:

$$Q^\pi(S_t, A_t) \leftarrow Q^\pi(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q^\pi(S_{t+1}, a) - Q^\pi(S_t, A_t)] \quad (2.1)$$

In practice, since Mnih et. al.'s work on Atari [\[37\]](#), this algorithm has been updated to use a Neural Network for a function approximator of the action-value function $Q^\pi(s, a; \theta) \approx Q^*(s, a)$, also known as a Q-Network. To train a Deep Q-Network ([DQN](#)), the updates are done by minimising a loss function through gradient descent. Formally, for every gradient iteration i , the loss function $\mathcal{L}_i(\theta_i)$ is defined as:

$$\begin{aligned} \mathcal{L}_i(\theta_i) &= \mathbb{E}_{s, a \sim \rho(\cdot)} [(y_i - Q^\pi(s, a; \theta_i))^2], \\ y_i &= \mathbb{E}_{s' \sim P(s'|s, a)} [r + \gamma \max_{a'} Q^\pi(s', a'; \theta_{i-1}) | s, a], \end{aligned}$$

where y_i is the equivalence target for iteration i , and $\rho(s, a)$ is the *behaviour distribution* of the learned policy. Differentiation on the loss function has the following gradient:

$$\nabla_{\theta_i} \mathcal{L}_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot); s' \sim P(s'|s, a)} \left[\left(r + \gamma \max_{a'} Q^\pi(s', a'; \theta_{i-1}) - Q^\pi(s, a; \theta_i) \right) \nabla_{\theta_i} Q^\pi(s, a; \theta_i) \right]$$

It can be noted on the similarity between the gradient descent step of [DQN](#) and the update step of the baseline Q-Learning algorithm. The pseudocode for the Experience Replay [DQN](#) version can be seen in [Appendix A](#) as algorithm 1.

Deep Deterministic Policy Gradient

TD-Learning based methods like Q-Learning and [DQNs](#) require the extraction of the true policy through greedily picking the best action. This method works well for short discrete

action spaces, since they can be easily and reliably sampled. However, for a continuous action space, the Q-Learning approach would never converge well, since the greedy sample would be too large, and potentially inaccurate, since the continuous space will need to be discretised to a certain degree. Furthermore, given the implementation details of Deep Q Networks, the output layer would be unreasonably large, even for simpler problems like Continuous Cart Pole [30].

Based on the Deterministic Policy Gradient (DPG) algorithm [42], Lillicrap et. al. proposes the Deep Deterministic Policy Gradient (DDPG) [38]. The approach makes use of neural networks as function approximators for two components, instead of just one: an actor network $\pi(s; \theta^\pi)$, and a critic network $Q(s, a; \theta^Q)$. Intuitively, the actor network will produce the most likely action the agent should take, given its current state, and the critic will represent the approximate action-value function, used to assess the actor's action, given its state. An informal diagram is presented in figure 2.3.

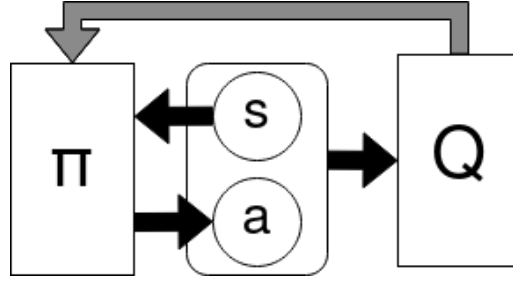


Figure 2.3: Actor-Critic interaction in a DDPG

To optimise the actor and the critic, instead of computing the values of the action-value function and finding the optimal action from those, the focus is shifted on the policy, or the actor, to act optimally. Formally, to update the critic, a similar approach to DQN is taken, only that the greedy action-value $\max_{a'}(Q(s', a'; \theta_{i-1}^Q))$ is replaced by the action taken by the actor policy $a = \pi(s; \theta_{i-1}^\pi)$:

$$\mathcal{L}_i(\theta_i^Q) = \mathbb{E}_{s, a \sim \rho^\pi(\cdot)} [(y_i - Q^\pi(s, a; \theta_i^Q))^2],$$

$$y_i = \mathbb{E}_{s' \sim P(s'|s, a)} [r + \gamma Q^\pi(s', \pi(s'; \theta_{i-1}^\pi); \theta_{i-1}^Q) | s, a],$$

and the actor network is then updated through gradient descent on the performance objective J :

$$J(\theta_i^\pi) = \mathbb{E}_{s \sim \rho^\pi} [R(s, \pi(s; \theta^\pi))],$$

$$\nabla_{\theta^\pi} J \approx \mathbb{E}_{s \sim \rho^\pi} [\nabla_a Q(s, a; \theta^Q)|_{a=\pi(s; \theta^\pi)} \nabla_{\theta^\pi} \pi(s; \theta^\pi)].$$

The pseudocode for DDPG can be found in Appendix A as algorithm 2. Note there are modifications to the logic, since in practice, the use of target networks is preferred for training stabilisation, compared to directly modifying previous iterations of the networks.

2.2.3 Training Stabilisation

Reinforcement Learning using function approximators has been notorious for its difficulty to ensure convergence and robustness, since as a baseline, when using Neural Networks, empirical data suggests that the theoretical unaltered training leads to policies which are

slow to converge and quick to be forgotten later in the training [11].

To minimise the harmful effects of random policy initialisation and gradient descent on edge cases in the environment, other than through simply updating the networks on batches of experience data, a few more creative approaches have been proposed in the literature, which do not come implicitly from standard Machine Learning theory. These methods, up to date, have not been proven formally to stabilise training. Yet, they have been shown to stabilise the training of RL algorithms through empirical means. In current SotA, it is now standard practice to include at least a variation of each of the following two methods: target networks, and experience replay buffers.

Target Networks

A discovery widely accepted as the state of the art in updating NN-based function approximators in RL is the use of a deep copy of each learnable network, called the Target Network.

The intuition follows from the way Q functions are being approximated during training. From equation 2.1, one can notice the usage of the value of the Q function to update itself. Assuming this is the method undertaken to update the DQN function approximator of a Deep Q Learning algorithm, it would mean that the value the DQN needs to approximate a value that is constantly changing, reminiscent of an imperfectly tuned PID controller, always one step behind the true value.

To counteract this instability, the usage of a *target network* $Q^{\pi'}$ is employed to replace the value of the original network. This target network is a deep copy of the original DQN, only it is not modified during each update, giving the original DQN time to "catch up" and give a better approximation of the true value function. The target network will only be updated every X amount of time steps, where X is a discrete hyperparameter left to be tuned by the engineer in charge with training the network. Alternatively, the update will be made in a "soft" way every time step [38], by modifying the target network with a small percentage $\tau \in (0, 1)$ of the updated original network.

The target network is used in a similar way by DRL algorithms other than DQN, if they require an estimation of their value in future time steps during their update. The algorithms in Appendix A are modified to use network update equations that estimate the cost functions with target networks, replacing the original equations of each algorithm.

Experience Replay Buffers

First proposed by Lin et. al. in 1992 [43], then popularised by Mnih et. al. [11], the Experience Replay Buffer represented the state of the art of stabilisation methods for RL at that time. The buffer works by storing, at training time, the set of past experiences $\langle s_t, a_t, r_t, s_{t+1} \rangle$ for every time step t , then learning in each update step over a random batch of sampled data. Intuitively, this means that the agent is learning not just from its most recent experiences, assuming they represent better performances, but also from its older experiences, ensuring previous mistakes do not repeat themselves. This assumption was shown to hold well empirically, with many successful algorithms using it for training stabilisation [44, 38, 45], and ablation on the replay buffer showing decreased performance.

In practice, the experience replay is unable to hold on to all experiences the agent went through, and even if it did, the sample size usually stays the same during training, leading to a decrease in the efficacy of the buffer over time, and potentially to forgetting interme-

diates policies. To minimise this effect, other than by limiting the size of the buffer and inserting/removing sets of experiences in a FIFO fashion, methods to maintain the most relevant episodes were introduced, most notably the Prioritised Experience Replay Buffers [46], which introduced a novel way of sampling the experiences, based on their significance.

With that said, most of what experience replay does is that it keeps track of episodes which went well, and episodes which did not. For optimisation-based scenarios, like safe and fast driving on a highway, this is not a problem. However, for goal-based environments, there might be more to learn, which is not explained by the buffer. To tackle this limitation, OpenAI introduced Hindsight Experience Replay (HER) [47] as a novel method of dealing with sparse rewards. The replay buffer gets populated with a new term, called the goal g , which gets stored with every experience of an episode. At the end of the episode, if the given goal is not achieved, the buffer also gets populated with one or multiple copies of that episode, in which the rewards are modified to reflect a scenario where the pursued goal is replaced by the goal achieved, intuitively turning the failed episode into a win for an alternative scenario configuration. This modification takes some pressure off the RL algorithm and off the reward engineering, but places it instead on the goal definition, and on the re-definition in case of a failed episode. Overall, HER shows the power of the replay buffer as a flexible tool, which can be used not only for stabilisation, but for robustness, since the HER was observed to reach its goals on validation even when the agent did not have great success in training.

2.3 Multi-Agent Reinforcement Learning

Single-Agent Reinforcement Learning has been an extremely powerful tool for solving dynamic problems, from simple test scenarios [30], to surpassing professional human players in competitive games like Go and Chess [12], to eventually reaching human level dexterity in real life robotic manipulation tasks [48].

However, for most of the aforementioned tasks, there exist a few assumptions which are not necessarily true, or at the very least impractical to ensure, for the majority of real life scenarios. Firstly, there is an assumption that the entirety of the working state is observable to the agent in the environment. Secondly, a single agent is taking a single action every time step (even in adversarial scenarios). Finally, that future states are independent of past states, also known as the Markovian assumption.

In Multi-Agent Reinforcement Learning (MARL), these assumptions are mostly relaxed. In MARL, the target for learning is a Multi-Agent System, where cooperative, competitive or mixed agents attempt to solve a problem where they share the environment. In such a system, all agents take their actions concurrently, and hope to optimise their own reward function. Note that this reward function can be shared in the case of cooperative agents [49].

2.3.1 Partially Observable Markovian Decision Process

In both single-agent and multi-agent settings, it is likely for the agent(s) to not have full knowledge of their environment at every time step. This is a practical consideration, since an agent in the real world would frequently have imperfect and incomplete sensory information of its environment. Uncertainty would normally break the MDP assumption of full environment state knowledge, which previously ensured convergence properties for learning agents. In such scenarios, the Partially Observable MDP (POMDP) was proposed as

an extension of the Markovian Decision Process, first as a solution to solve spoken dialog systems [50], then further exploited for decentralised multi-agent systems [51, 52, 53].

The **POMDP** is dealing with uncertainty in environments through the concept of observability. That is, the agent no longer has complete knowledge of its environment, but a *belief* of the state of the world [54]. The **POMDP** is formally defined by introducing two new terms to the **MDP** tuple presented in the previous section: $\langle S, A, \Omega, P, O, R, \gamma \rangle$. In the new tuple, Ω is the set of possible observations, and $O \in \mathcal{M}(\Omega)^{S \times A}$ is the dynamics function in terms of the probable returned observation, given the current state and action. The other terms maintain their definition from the **MDP**, at least for the single-agent case.

The agent's interaction with the environment in a **POMDP** can be visualised in figure 2.4a, where the only notable difference is the processing of observations instead of direct states from the world. Furthermore, the agent's belief of the state of the environment at a discrete time step $b(s_t)$ can be formalised as a probability distribution [54]:

$$\forall s_t, b(s_t) \stackrel{def}{=} Pr(s_t | o_t, a_{t-1}, o_{t-1}, \dots, a_1, o_1, a_0).$$

It is worth noting, based on this equation, that an agent with sensory input of the environment will have better awareness of the overarching state when it can make use of its previous states and actions; further breaking away from the Markovian assumption of independence from the past.

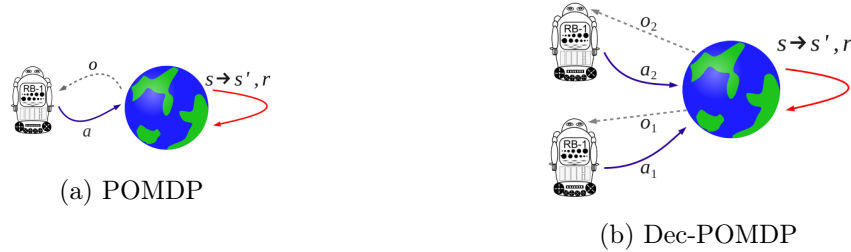


Figure 2.4: Agent-Environment interaction in a **POMDP** [54]

For the Multi-Agent scenario, another extension is introduced to the **MDP**, called a Decentralised-**POMDP** (Dec-**POMDP**) [54], with the purpose of introducing joint dynamics, actions and observations, based on multiple independent agents. Formally, the Dec-**POMDP** tuple introduces only one more element to the **POMDP** tuple, but also redefines some of the pre-existing ones. The new definition is $\langle N, S, A^i, P, R, \Omega^i, O \rangle$, where N is a finite set of agents, S is the set of possible environment states, A^i is the set of actions for agent i , $P \in \mathcal{M}(\Omega)^{S \times A}$ is the dynamics transition function, where $A = \times_{i \in N} A^i$ and $\Omega = \times_{i \in N} \Omega^i$ are the composed sets of the actions and the observations of all agents $i \in N$ respectively, $R : S \times A \times S \rightarrow \mathbb{R}$ is the reward function of the environment and Ω^i is the set of observations for agent i and $O \in \mathcal{M}(\Omega)^{S \times A}$ is the set of observation probabilities. Note that in the case of competitive environments, the reward function can be defined as $R : S \times A \times S \rightarrow \mathbb{R}^n$, where R is now split into multiple functions $R_i : S \times A \times S \rightarrow \mathbb{R}$, each being the the reward function of a single agent i .

2.3.2 Centralised Training Decentralised Execution

The benefits of deploying Multi-Agent Systems come at execution time, in the form of faster, more scalable architectures [33, 55], as each agent can make, in theory, its own decisions based on the observations it is exposed to, as opposed to waiting to aggregate all

state information to a central controlling agent. However, not having information about the actions and observations of other agents at training time can be detrimental to the training time of cooperative, and even competitive agents, leading to slow convergence towards a non-ideal local optimum.

Instead, recent developments of [MARL](#) algorithms have focused on a framework called Centralised Training Decentralised Execution ([CTDE](#)) [[56](#), [57](#), [58](#)]. The general structure of the framework is that, at training time, an aggregation of the observations and actions of all agents in the environment takes place, where more accurate representations of the state-value and action-value functions can be generated.

This paradigm is represented in the literature by Value Decomposition Networks [[59](#)] or QMix [[57](#)] approaches of centralisation for training. The main consideration of either approach is the balance between training speed and credit assignment, where how to ensure the latter is still an ongoing optimisation question [[60](#)].

Multi-Agent Deep Deterministic Policy Gradient

One of the first stable adaptations of Actor-Critic algorithms to [MARL](#) using the [CTDE](#) framework was the Multi-Agent [DDPG](#) [[56](#)]. The naïve approach for applying [DDPG](#) to a [MAS](#) directly would be to create an actor and a critic for each agent (or to share it, if we assume homogeneity). For reference, this can be referred to as Independent [DDPG](#) ([IDDPG](#)). As discussed above, this can lead to sub-optimal returns from the critic and, respectively, an actor which will converge to a local optimum.

Assuming a Dec-[POMDP](#) framework, [MADDPG](#) maintains actors defined as $\pi_i(o) : \Omega^i \rightarrow A^i$, the same as for [IDDPG](#). The difference comes in the way the critics are defined: $Q_i^\pi(x, a_1, \dots, a_N) : (\times_{i \in N} \Omega^i) \times (\times_{i \in N} A^i) \rightarrow \mathbb{R}$, named *centralised action-value function*. The state information here is defined as $x = (o_1, \dots, o_N)$, however, further information can be introduced in x , based on what is available or relevant in the training phase to measure. A diagram of the architecture can be seen in figure [2.5](#).

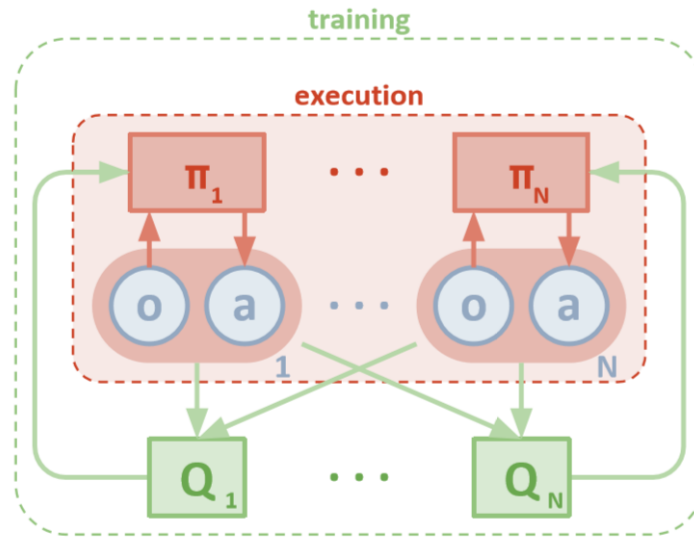


Figure 2.5: Overview of [MADDPG](#) architecture [[56](#)]

Formally, when using Deep Neural Networks as function approximators, the change in the definition of the critic results in different losses and gradients for the updates of both

the critics and the agents:

$$\begin{aligned}\mathcal{L}(\theta_i^Q) &= \mathbb{E}_{\mathbf{x}, a, r, \mathbf{x}'} [(Q_i(\mathbf{x}, a_1, \dots, a_N; \theta_i^Q) - y)^2] \\ y &= r_i + \gamma Q'_i(\mathbf{x}', a'_1, \dots, a'_N; \theta_i^Q) |_{a'_j = \pi'_j(o_j; \theta_i^\pi)} \\ \nabla_{\theta_i^\pi} J(\pi_i) &= \mathbb{E}_{x, a \sim D} [\nabla_{\theta_i} \pi_i(o_i; \theta_i^\pi) \nabla_{a_i} Q_i(x, a_1, \dots, a_N; \theta_i^Q) |_{a_i = \pi_i(o_i; \theta_i^\pi)}]\end{aligned}$$

Note that since the updates are conditioned on the aggregation of all actions of all agents, the environment maintains stationarity even if the policies change: $P(s'|s, a_1, \dots, a_N, \pi_1, \dots, \pi_N) = P(s'|s, a_1, \dots, a_N) = P(s'|s, a_1, \dots, a_N, \pi'_1, \dots, \pi'_N)$, for any $\pi_i \neq \pi'_i$. The pseudocode can be found in Appendix A as algorithm 3.

2.3.3 CTDE Extended

One thing which complicates multi-agent learning using the CTDE approach is the centralised critic. Not only is the centralisation of all information a difficult approach during training in decentralised environments, especially when considering real-life robotics applications, but the critic itself is a complex element to stabilise robustly in scenario variations, particularly when the number of agents is not stationary.

In many real-life applications, multi-agent control is a tool to ensure robustness of the system in circumstances where the number of agents is fluctuating, either by randomly joining or leaving the system, or through partial system failure, when some agents become disconnected [61]. The majority of the literature on MARL for CTDE is focused on a centralised critic, assuming the same number of agents throughout training, leading to difficulty in generalisation for either more or less agents. Furthermore, contrary to the assumption that homogenous agents should be easier to train and stabilise, MADDPG and MAPPO are designed with heterogeneity of the agents in mind. This means that the order of the observation and state input is actually important during training, leading to certain inefficiencies and inconsistencies in the output of the critic Q network when tackling problems containing homogenous agents with the baseline models.

To counteract that, based on CTDE, another type of architecture started emerging in the literature, with a separate critic function for each agent $Q_i(o_i, a_i; \theta_i^Q)$, and a main aggregator critic $Q_{tot}(o, a)$, called *utility functions* and *value function* respectively. The networks which proposed and influenced this architecture most were Value Decomposition Networks [59] and QMix [57]. Structurally, the new architecture proposed is a useful baseline, as the resulting representation of the separate critic approximations can be reasoned about using approaches which incentivise better scalability on multiple agents.

Multi-Agent Attention Critic

By itself, the alternative CTDE architecture is not enough to lead to better scalability in terms of the number of agents at training time. The architecture needs to be combined with a form of learning mechanism which is more invariant to input size differences, like RNNs, CNNs or Attention. One particular development focusing on MARL developed an algorithm called Multi-Agent Attention Critic (MAAC) [62].

Intuitively, MAAC is enhancing each agent's critic Q_i value function with attention to the other agents' observations and actions, compared to its own, leading to better self-awareness in social settings, and to learning to extract relevant detail for any number of

agents it interacts with, an especially useful skill in homogenous environments. Formally, the actor network does not change, and the critic is redefined as

$$\begin{aligned}
Q_i(o, a; \theta_i^Q) &= f_i(g_i(o_i, a_i), x_i) \\
x_i &= \sum_{j \neq i} \alpha_j v_j \\
v_j &= h(V e_j) \\
\alpha_j &\propto \exp(K_j^T Q) \\
K_j &= W_k e_j \\
Q &= W_q e_i \\
e_i &= g_i(o_i, a_i),
\end{aligned}$$

where f_i , g_i and h_i are simple multi-layer perceptrons (MLP) function approximators, x_i is the weighted sum of each agent's value v_i , where the weight α_j represents the attention filter given by the query Q and key K of the attention mechanism. The queries Q and the keys K_j are just the weighted embeddings of the agents they represent. We will refer to agent i as the ego agent. The weights $\langle W_q, W_k, V, \theta_i^Q \rangle$ represent training parameters.

For MAAC, the actors and critics are updated through slightly modified versions of the Advantage Actor-Critic loss functions:

$$\begin{aligned}
\mathcal{L}_Q(\theta^Q) &= \sum_i \mathbb{E}_{\langle o, a, r, o' \rangle \sim D} (Q_i(o, a; \theta_i^Q) - y_i)^2 \\
y_i &= r_i + \gamma \mathbb{E}_{a' \sim \pi'(o'; \theta^{\pi'})} [Q'_i(o', a'; \theta_i^{Q'}) - \alpha \log(\pi'_i(a'_i | o'_i; \theta_i^{\pi'}))] \\
\nabla_{\theta_i^\pi} J(\pi_i) &= \mathbb{E}_{o \sim D, a \sim \pi} \left[\nabla_{\theta_i^\pi} \log(\pi_i(a_i | o_i; \theta_i^\pi)) (-\alpha \log(\pi_i(a_i | o_i; \theta_i^\pi)) + A_i(o, a)) \right],
\end{aligned}$$

where $A_i(o, a)$ is the advantage function, formally defined by computing a baseline b as follows:

$$\begin{aligned}
A_i(o, a) &= Q_i(o, a; \theta_i^Q) - b(o, a_{\setminus i}) \\
b(o, a_{\setminus i}) &= \mathbb{E}_{a_i \sim \pi_i(o_i; \theta_i^\pi)} Q_i(o, (a_i, a_{\setminus i}); \theta_i^Q)
\end{aligned}$$

The pseudocode for MAAC is present in Appendix A as algorithm 4.

2.4 Learning Mechanisms

In theory, a sufficiently large neural network function approximator, given an infinite amount of time, can perfectly represent any possible function or policy. However, it is impractical to depend on this axiom for training convergence, and it can also be the case that a function approximator converges to a local optimum, or that it starts forgetting. To circumvent the unreliability of baseline neural networks in practice, different learning mechanisms are employed, inspired by real life learning and psychology, to help stabilise learning, but also to provide information and perspectives which otherwise would be lacking from the baseline neural network.

2.4.1 Recurrent Neural Networks

The Recurrent Neural Network (RNN) is a mechanism applied to neural networks which allows the storage of state information from previous iterations of the network, to allow the simulation of dynamical systems with discrete time. Intuitively, the network maintains a localised memory medium h_{t-1} , which is accessed and modified every time step t with knowledge from the new input to the network x_t by a state transition function f_h . This updated information h_t is then used to retrieve the output y_t by an output function f_o . Formally:

$$\begin{aligned}\mathbf{h}_t &= f_h(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta_h) \\ \mathbf{y}_t &= f_o(\mathbf{h}_t; \theta_o)\end{aligned}$$

where the state transition function f_h and the output function f_o are neural network function approximators.

Two particular extensions of this model, the Long Short-Term Memory (LSTM) [63] and the Gated Recurrent Unit [64] have represented the state of the art on discrete-time dynamic tasks, like speech recognition in NLP, until the introduction of the attention transformer [65]. They still represent a highly intuitive way of recalling state information through time, which is a great asset in dynamic reinforcement learning environments.

2.4.2 Attention

The Attention Mechanism represents a way for algorithms to model and learn, from given input, on what features it is worth focusing the most. In other words, what part of the input is most significant. The mechanism has been proposed as early as 2014 by Bahdanau et. al. [66], where it was introduced to the field of Natural Language Processing (NLP) as an enhancement over the RNN-based encoder-decoder architecture, with limited performance improvements. The Attention Mechanism received more acknowledgement once the Transformer architecture was introduced in 2017 [65], when the RNN was removed from the encoder-decoder architecture and significant performance improvements were observed for translation tasks. This incentivised further research, leading to the current State-of-the-Art (SotA) to be led by models built on top of attention mechanisms, not only for NLP tasks [67, 68, 69], but also for computer vision (CV) [70, 71, 72, 73] and reinforcement learning tasks [74].

Intuitively, the attention mechanism can be described as the extraction of a value from a key-value mapping, where a query Q is used to extract the value V out of the mapping with the key K being most similar to the query. In practice, the output is a weighted sum of the values, where the weights are given by the cosine similarity between the query and the key QK^T , thus giving the formal attention function definition

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V,$$

where d_k represents the size of matrix K , and is introduced as a scaling factor to prevent the vanishing gradient problem [65], and the $softmax(.)$ function is added to make the scaled weights sum up to 1. A diagram of the equation can be observed in figure 2.6a

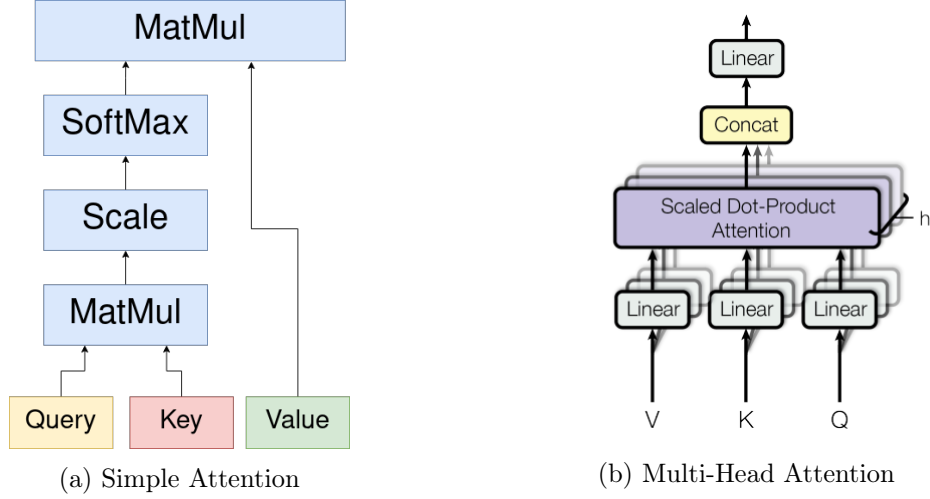


Figure 2.6: Attention Mechanism in the Transformer Architecture [65]

Self-Attention and Multi-Headed Attention

A powerful baseline model for the usage of attention mechanisms is Self-Attention [65], where the queries, keys and values are represented as linear embeddings of the same value, that usually being the input of the model (i.e. a sentence for NLP, an image for CV, the observation for a RL agent). Intuitively, a head of self-attention will lead to the output of the layer to focus on a single, but clear aspect of the input, disregarding everything except for the elements which are similar to each other. In a trainable model, self-attention is a powerful tool, as it gives more significance to similar elements, like words in a sentence, or shapes in an image.

On the other hand, by itself, it is possible for self-attention to overfit a certain set of features which might not be as relevant to the problem as previously considered. To overcome this problem, Multi-Headed Attention is proposed. Instead of focusing on a single attention-mechanism, multiple heads can be instantiated in parallel, with different parameters, in the hopes that each head will focus on a separate feature of the input, leading to a concatenation of all significant features over which the rest of the model can process. A diagram of the multi-headed attention model is given in figure 2.6b.

Social Attention

A more specialised type of attention mechanism can be used in dynamic environments with multiple actors, like reinforcement learning. It is called Social Attention [75], and it attempts to learn similarities between the ego-agent and other actors in a social environment. This is achieved formally by replacing the query with the state/observation features of the ego-agent $Q = [o_0]$, and the keys with the state/observation features of the other social agents in the environment $K = [o_1, \dots, o_N]$, where the state of the environment can be seen as the matrix of all observations $O = [o_0, \dots, o_N]$. The value can then be left as a simple linear projection of the state features $V = f(O)$, where $f(\cdot)$ is an MLP.

Some of the goals this mechanism achieves is an agent model robust to a varying degree of input sizes, as well as permutation invariance for the states of the other actors in the input. As any other attention model, multiple heads of Social Attention can aid in the extraction of separate common features between the ego-agent and the other social agent, like common goals in cooperative or adversarial scenarios.

2.5 Related Work

In the following section, a quick summary and overview of related work and relevant literature will be presented. Compared to the sections above, which were focused on the theoretical foundations necessary for the undertaking of this work, this section highlights the practical applications which were developed prior to this thesis, with various degrees of overlap between the concepts. The primary purpose of this section is to provide the reader with the general perspective of the state of the art, practices, as well as observed limitations which have either been tackled during the undertaking of this dissertation, or have been highlighted for future work.

2.5.1 Robustness in Reinforcement Learning

The work of developing autonomous agents to be tolerant to disruptions or failure started before reinforcement learning became a popular approach of solving tasks, with some early work focused on multi-legged robots with malfunctioning appendages [76, 77]. A more recent continuation of this work was the T-Resilience algorithm of Koos et. al. [78], which tackled the problem of a hexapod's leg being cut in half by introducing self-perception and model-based reinforcement learning. The intuition is that, given partial knowledge of the dynamics of the environment, as well as the awareness of its own dynamics and current limitations through self-diagnosis, an agent is able to re-plan its behaviour in order to continue pursuing its task. This work, however, assumes that the initial goal is still feasible after the said fault gets introduced in the system (in this case, simply moving forward) and that the environment is safe enough to permit real-life validation of the policy trained on the planning model which the agent trains on. Both assumptions need to be thoroughly considered in risky situations, like search-and-rescue missions, when attempting to re-adapt to solve the goal. Cully et. al. continued on this line of work by removing the need to do self-diagnosis [79] and speeding up the adaptation process significantly by preparing different behavioural policies a priori, and through "Intelligent Trial-and-Error", the policy with the highest performance is chosen. This last approach reduces the over-reliance on having a highly correct model of the environment at runtime or self-diagnosing mechanisms in place, and minimises the amount of time the agent needs to validate its best policy in reality. The "Intelligent Trial-and-Error", however, if given too much liberty, might put the agent in an unsafe state by the time the best performing policy is located. Applications of bi-level or constrained optimisation might solve this issue in the future.

As for robustness in simulated traffic scenarios, a highly relevant work by Li et. al., proposes a Fault-Tolerant Cooperative Control (FTCC) method, applied to intersections with continuous dynamics, to replan the trajectory of the faulty agents in order to achieve a different goal, based on the agent's capabilities after the fault occurs [80]. The current limitation of this work is that only the faulty agent is replanning its path after the fault occurs, to limit computation cost. Furthermore, the environment assumes perfect knowledge of the behaviour of the other agents, which is impractical to assume for many real life scenarios. Since it is possible for multiple faults to occur in traffic, which are not necessarily caused by one of the cooperative agents, the method can be relatively impractical due to its high computational cost and reliance on perfect knowledge.

2.5.2 Deadlocks in Multi-Agent Systems

The problem of resolving deadlocks in multi-agent systems is not new in literature, but the focus of work is in deadlock prevention rather than resolution on occurrence [81, 82, 83]. Particularly, the definition of the deadlock does not seem to allow its occurrence unless

the behaviour of the agents is deterministic, and the environment is discretised both in terms of state and time. Furthermore, most applications in the literature that consider the occurrence of a deadlock use a centralised controller. In such an environment, given the deterministic nature and consistency which the environment can ensure, the problem of preventing deadlocks is highly simplified. Therefore, the resolution of the deadlock is overlooked, due to the relatively high degree of complexity required, compared to the decreased to non-existent chance of a deadlock’s occurrence provided by the focus on prevention.

There has been work done on deadlock resolution, even for potentially non-deterministic RL algorithms [84]. This work focuses on maze solving by multiple agents, and a deadlock can occur if two agents find themselves in the same space on a grid at the same time. The deadlocks were made solvable by augmenting the state of an agent in a potentially deadlocking scenario with the state of the other agents which might get deadlocked, representing a form of probabilistic inference bootstrapped by a shield [85]. This strategy proved to be effective in the given environment, and provides insight into the potential of shared observations, communication and shielding in multi-agent systems, and it is shown to work on agents trained with reinforcement learning, an aspect which seems to be overlooked in deadlock resolution literature. However, this work maintains the assumptions of discretised state and time, as well as a centralised controller for the purpose of detecting the potential deadlock. Furthermore, it assumes knowledge of the behaviour of the other agents, which cannot always be the case in heterogenous environments.

Deadlocks in AGV Systems

Autonomous Guided Vehicles (AGV) have already been deployed for almost two decades, in warehouse management [86], in ports for cargo handling [87], with prospects of being deployed in mining operations in the near future [88]. For all these problems, there is extensive literature into the detection, prevention and resolution of deadlocks [28], with industry making use of the theory in practical applications.

Given the survey in [28], there are a couple of limitations to the current state of the art for AGVs which make some of the theory difficult to transfer from industry-standard AGV to continuous traffic scenarios with AVs. First and foremost, the control aspect of the agents in AGV systems is, in general, centralised, having perfect knowledge of the states and observations of autonomous agents within the system. This is an important asset in fully autonomous environments, which benefit from deterministic hardware and software safety features, robust infrastructure and a deployment area relatively small in terms of number of agents and area of effect. However, the majority of traffic environments are city-wide, with little to no infrastructure in place yet, making information aggregation a significant overhead in terms of time and data processing, in an environment infamous for its dependence on fast reactions and quick replanning for safety to be maintained.

Furthermore, most of the practical applications of AGV system planning and deployment model the state and coordination policies in terms of abstractions that either discretise, or otherwise simplify the state and action space of the problem. This has the benefit of making many decisions more computationally feasible in a shorter amount of time. The downside is that all the aforementioned benefits are physically constrained by the quality of the infrastructure for deployment, which needs to provide the central controller unit with the correct measurements of the abstracted states and actions. When AGV systems are deployed, they are built with the infrastructure in mind, through the setup of rails, belts or lines to be followed by the agents. The last aspect has both upsides and downsides, since it provides faster central control and safety at the expense of partial performance degra-

dation, and introduction of deadlocks that are more difficult, or impossible, to resolve.

In summary, AGV systems have been well developed for around two decades, and have been deployed successfully in industrial applications, with deadlock dealing capabilities. These capabilities, however, are not trivial or practical to transfer directly in real life AV traffic scenarios, due to the significant differences in the infrastructure and abstractions used to define the states of either environment in practice.

Traffic Deadlocks: Gridlocks

Gridlocks is a term synonymous to deadlocks in traffic, and they occur when multiple agents find themselves in a road network configuration which does not allow agents to move on their trajectory anymore. As for deadlocks in scheduling theory, the majority of work focuses on the avoidance of their occurrence [89, 90, 91], in environments with deterministic agent behaviour. There are exceptions with focus on the resolution of such deadlocks on occurrence [92, 93], but otherwise, through the work done for this thesis, no evidence was found of related work which attempts to resolve gridlocks with RL-based optimisation methods.

2.5.3 Deep Reinforcement Learning for Autonomous Driving

The developments in the field of RL over the past 10 years have significantly enhanced the capabilities and potential of autonomous driving in real life [1], from better perception and sensory comprehension [2, 94], to better behaviour control and path planning capabilities [95]. Vehicles with varying degrees of SAE autonomy [96] are already deployed by manufacturers like Tesla and Waymo [4], and infrastructure is being tested in preparation for the introduction of cooperative vehicle fleets [3].

Multi-Agent Learning

Up to date, the majority of multi-agent reinforcement learning for traffic scenarios has been done in simulators [97, 98, 29], with a few works using real life robotic environments [99] to validate the coordination aspect of the work [100].

One particular relevant work by Toghi et. al. used the *Highway-Env* traffic simulator [29] to model agents which are capable of sympathising with potential human drivers, for the purpose of modelling a fair way of solving the highway merging scenario [101]. This work seems to make use of agents trained with independent DQNs, with a bird's eye view of the highway scenario as the observation space. The major contribution seems to be in the way the reward function is being modelled, being hyperparametrised on a degree of ego, cooperative and sympathetic performance, the last one being represented by the performance of the humans in the system. The results show that various degrees of cooperation can be modelled through the use of highly specialised reward methods, even with agents who are not controlled by the same policy. However, the scope of the work is limited to independent Deep Q Learning, so the non-stationarity of MAS is not directly circumvented other than through the use of various learning mechanisms and stabilisation methods.

2.6 Background Key Takeaways

Deadlocks are a concept shared in the literature of multi-threaded programs and multi-agent systems, and they represent a cyclical waiting queue for resources occupied by waiting

processes. The research into methods for the prevention of deadlocks is considered more valuable in general, compared to research into deadlock resolution, due to the robustness which it can bring to a system and, consequently, better performance on average, due to less reliance on either deadlock resolution or runtime resets. For multi-agent systems, the majority of applications of deadlock resolution theory are in discrete-space environments, or other environments with constrained dynamic models, with little to no consideration for deadlocks to exist in continuous spaces.

In recent years, there has been a resurgence of interest into [MARL](#), due to the discovery of the [CTDE](#) architecture. Much of the research has been focused on trying to adapt the existing algorithms from [RL](#) to the multi-agent domain. The vast majority of these adaptations were relatively simple to achieve, since they represented a direct extension of the already existing Actor-Critic architecture of its single-agent counterpart, like [MADDPG](#) [38, 56] and [MAPPO](#) [102, 58], which is a sufficient difference to counteract the non-stationarity problem of their underlying Dec-[POMDP](#) structure. However, in their base form, these algorithms have become notoriously difficult to stabilise, leading to the necessity of extensions with the main purpose of improving their convergence limitations [103, 104].

Notably, alternative architectures emerged by either modifying the overall structure of the [CTDE](#) framework, like [VDN](#) and [QMIX](#) [59, 57], but also significant enhancements to the previous ones with useful learning mechanism like [RNNs](#) for [R-MADDPG](#) [103] and Attention for [MAAC](#) [62]. The former provides better hindsight-awareness, whereas the latter provides more clarity to the function approximators as to which features are the most important, as well as enhance the critic to allow a fluctuating number of actors at training time.

Overall, the literature for [MARL](#) has matured enough to start being applied on not just optimisation-based tasks, but also on tasks which are more focused on safety, like rebound and damage mitigation, with the hope of achieving greater robustness for unexpected scenarios.

Chapter 3

Methodology

This chapter provides an overview of the majority of the contributions in this thesis. The setup of the development environment will be presented, in terms of the simulators and [RL](#) libraries which were experimented and considered for this project. Then, the *rebound* optimisation goal is described, in relation to the rest of the literature focused on safety. Then, the formalisation of the deadlock in a [MAS](#) is defined and put in context with rebound problems and traffic deadlocks. Then, the design of the simulated deadlocked intersection environments is motivated. Finally, the implementation of the evaluated multi-agent reinforcement learning algorithms is described.

3.1 Development Setup

In the development and experimentation, the following tools were used:

3.1.1 Simulator Survey

To run the experiments, a significant portion of time was spent to pick the simulator most suitable to the scope of the project. Direct experimentation in the environments was done within DuckieTown Gym [\[105\]](#), SUMO [\[97\]](#), SMARTS [\[98\]](#) and Highway-Env [\[29\]](#).

DuckieTown Gym was the first one considered, due to its integration with the DuckieTown environment and real life Duckiebots [\[99\]](#). This would have been one of the better simulators for sim2real projects, as any development within the DuckieTown framework could be simpler to reproduce by other research teams, due to the open source nature of the software and the standardised simulated and real life environment. However, the scope of the problem is more focused on the emergent behaviour of the agents, and not necessarily the transfer of the behaviour to real life. Therefore, due to the largely unsorted complexity overhead of the current code base, related to the connection between simulated and real agents, *DuckieTown Gym* was dropped.

SUMO and SMARTS were considered afterwards, due to their high current usage in the literature [\[97\]](#) and high potential of growth in the research communities [\[98\]](#). SUMO has the disadvantage of not being made with [RL](#) in mind, and is explicitly not designed to model fine grained control of vehicles (i.e. cars cannot move backwards). The former issue could have been circumvented with *Flow* [\[106\]](#), a library designed to extend the capabilities of SUMO to [RL](#); yet, this library's source code has not been maintained for years at the time the work for this thesis has been undertaken. The latter issue would have required a shift in this dissertation's scope, since it would have been impossible to express the "rebound from deadlock" behaviour this work is trying to incentivise. For

these reasons, *SUMO* has been considered out of scope for the current project. SMARTS was disconsidered for similar reasons, since it uses SUMO as a backend.

Finally, *Highway-Env* was considered, and it remained the main simulator for this project. Since it is designed with reinforcement learning in mind, the simulator is lightweight and provides many useful features for this project, most important of which is the inclusion of multi-agent observation and action spaces. Furthermore, it is compatible with reinforcement learning libraries like *StableBaselines3* [107] and, due to its simple design, it is easy to extend its capabilities and create new environments, as they all extend *OpenAI Gym*'s base Gym environment [30].

3.1.2 Reinforcement Learning Library

A short number of reinforcement learning libraries were considered, since the compatibility with the simulator was an important factor. Among those, there were *RLlib* [108], *StableBaselines3* [107] and *Rl-Agents* [31].

RLlib would have been the closest to the scope of multi-agent reinforcement learning, since most of the algorithms were implemented already. However, since *RLlib* works on a modification of the OpenAI Gym, it meant that the environments themselves would have needed a potentially large amount of tuning to make them compatible. Furthermore, due to the high complexity of the code base, further modifications to the learning algorithms themselves would have been impractical, especially for the case where inter-agent communication or hindsight experience needed to be introduced.

StableBaselines3 was initially considered due to its compatibility with the *HighwayEnv* simulator [29], its simplified design compared to *RLlib*, as well as the large community maintaining its code base. Its main downside has been that it was not designed with multi-agent reinforcement learning in mind. A significant amount of time was spent tackling the challenge of adapting the code to the multi-agent setting of HighwayEnv, changes which are still maintained in a personal fork¹ of the repository. However, the benefits of the centralised code base started to be outweighed by the overhead of each modification in the code.

Finally, *Rl-Agents* was considered, and is currently the library in which the majority of the contributions have been made. *Rl-Agents* is the library with the highest compatibility with the Highway-Env simulator, since it is designed by the same maintainer with this purpose in mind. Compared to *StableBaselines3*, there were fewer variations of RL algorithms implemented in it. Therefore, as one of the contributions of this dissertation an implementation of the MADDPG algorithm was written within the *Rl-Agents* framework, configurable with multiple attention variations. This algorithm is further discussed in section 3.5.

3.1.3 Visualisation Tools and Data Collection

For both tasks, Tensorflow's [109] *TensorBoard* tool was used, due to its simplicity and practicality.

¹<https://github.com/TibiGG/stable-baselines3>

3.2 Rebound from Failure

In this section, the identification and proposal of *Rebound* from failure as a safety feature for reinforcement learning is detailed, motivated and defined.

3.2.1 The Mid-point between Robustness and Damage Mitigation

The literature of safety in autonomous learning approaches has been split into two trends. The first one, *Robustness*, focuses on the idea that damaging situations should be avoided at all cost, placing safety as the primary objective of a problem, or at least, as a constraint objective. This approach allows for agents to ensure that, most of the time, damaging situations are avoided, at the detriment of performance. However, overemphasis on this aspect of safety can make agents overtrain on the aspect of avoidance, to the point where they do not have fine tuned behaviour in states with a higher degree of risk, as they are prevented or avoided during later stages of training.

The second branch of safety, damage mitigation or fault tolerance, focuses on the moments after the occurrence of a catastrophic, or a specific kind of failure, and how would the agent(s) behave to still complete their goals, as best as they can [110]. An example of this trained behaviour would be the adaptation in policies of robots after suffering partial damage to their structural integrity.

These two branches, both important in their own right, tend to overlook a significant inflection point in the structure of an accident’s occurrence. Robustness focuses on the planning and preparation for damaging situations by avoiding reaching the initial failures in a system, to prevent a cascade of failures which become increasingly more difficult to recover from. The danger of this focus is that, should one or multiple initial failures occur by chance, there is little damage control which the agent is prepared to do afterwards. Damage mitigation methods, in general, train agents in the aftermath of a damaging failure, when the system supposedly reached a state where further damage is improbable. This scenario is giving agents little to no time to reflect on the immediate actions that could have been taken to either prevent the accident, or at least mitigate part of the damage. Furthermore, other than some work on faults in traffic scenarios [80], there is little work focusing on the prevention of more accidents after the introduction of a fault, or returning to a normal state.

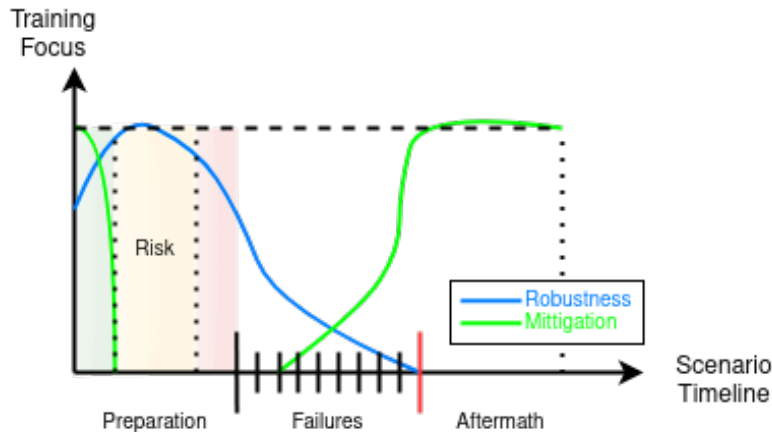


Figure 3.1: Graphic of Current Training Focus in Accident

For a general intuition of the literature’s development in terms of focus, a highly spec-

ulative graphic is proposed in figure 3.1, with a rough outline of the expected focus on training of the different "training for safety" approaches, given a scenario where potential accidents could occur. The area of importance is the one in between the systematic failures. It can be argued that accidents do not occur immediately, but are caused by multiple, consecutive smaller failures. For example, a car crash is not just the accidental hit, but the maneuver that led to a collision course, then the failure to make further maneuvers to avoid it. An accidentally crashed plate is not just the collision of the plate with the floor, but the dropping of the plate from a high place, then failure to catch said plate from the air, then failure to put something soft in between the plate and the ground in time.

Robustness, through damage prevention and avoidance, highly optimises on the states of medium to low risk, where there is a high probability of preventing the first failing states, at the cost of sacrificing preparation for meeting the high risk situations. Tolerance, through mitigation, highly optimises on states in the aftermath of an accident, where there is little to no risk involved, in order to make the occurrence of specific failures inconsequential to the reaching of the end goal of the environment.

Rebound from failure is proposed as an important area of research into safe learning for autonomous agents. It proposes a specialisation of the fault recovery problems, currently underexplored in the reinforcement learning literature, with the purpose of filling in the gap between *robustness* and *mitigation*. Further research in this area should help in the design of autonomous agents which, given a post-failure situation where critical failure becomes likely, the agent is able to react in a risk-aware manner that permits the possibility of returning to a normal state, without the use of predefined deterministic routines.

3.2.2 Definitions

Failure

For the purpose of rebound, failure in an environment is defined by the process of accidentally reaching a state from which optimising the goals becomes more difficult than it was at the beginning. That is, before reaching said state, the expected return at the beginning of the episode was higher than the new potential return. Formally, for an environment with dynamics $P(s'|s, a)$, given an agent with a trained policy $\pi(a|s)$, for an episode starting from state $s_0 = s$, if a failure occurs at time t_F , the occurrence can be formulated as:

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right] > \sum_{t=0}^{t_F} \gamma^t R(s_t, a_t) + \mathbb{E}\left[\sum_{t=t_F+1}^{\infty} \gamma^t R(s_t, a_t)\right], \quad (3.1)$$

where $s_0 = s, a_t \sim \pi(a_t|s_t), s_{t+1} \sim P(s_{t+1}|s_t, a_t)$

For example, this failure event could be the moment a cooperative traffic agent cuts in front of the path of another agent, or a fragile object slipping from one's hand.

Catastrophic Failure

Similarly to above, catastrophic failure is an event which triggers a lower expectation in the return value of the environment, only that this form of failure dramatically decreases the expected return, and can potentially lead to the goal of the environment to be unreachable. It is the sort of state which, at training time, common reinforcement learning practices would reset the environment after its occurrence. Formally:

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right] \gg \sum_{t=0}^{t_F} \gamma^t R(s_t, a_t) + \mathbb{E}\left[\sum_{t=t_F+1}^{\infty} \gamma^t R(s_t, a_t)\right] \quad (3.2)$$

For example, the collision of two vehicles, at the point where it damages the structural integrity of the vehicles involved, can be considered catastrophic, since it is unlikely for them to be able to continue to move, requiring special assistance and reparation to rebound. A similar case is the contact, after surpassing some threshold acceleration, of a fragile object with a hard surface, as this contact can lead to chipping or breaking.

Rebound

In the timeline of a scenario, if a failure occurs, it can be followed by other failure events in quick succession. For instance, a vehicle cutting the path of another vehicle could lead to collisions, to damage being done to the structural integrity of the vehicles involved, then to major traffic artery blocking. Also, the more a fragile object is left free-falling, the more probable the chance of said object to crash catastrophically to the floor, towards an unrecoverable state. For the first couple of failure events, it is possible to avoid the occurrence of the latter ones. However, if actions to prevent those states are not taken immediately, more failure events can occur, leading to eventual catastrophic failure.

$$\begin{aligned} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right] &> \sum_{t=0}^{t_{F_0}} \gamma^t R(s_t, a_t) + \mathbb{E}\left[\sum_{t=t_{F_0}+1}^{\infty} \gamma^t R(s_t, a_t)\right] \\ &> \sum_{t=0}^{t_{F_1}} \gamma^t R(s_t, a_t) + \mathbb{E}\left[\sum_{t=t_{F_1}+1}^{\infty} \gamma^t R(s_t, a_t)\right] \\ &\dots \\ &\gg \sum_{t=0}^{t_{F_k}} \gamma^t R(s_t, a_t) + \mathbb{E}\left[\sum_{t=t_{F_k}+1}^{\infty} \gamma^t R(s_t, a_t)\right], \end{aligned} \quad (3.3)$$

where subsequent failure incidents occur at time steps t_{F_0} to t_{F_k} , with potentially more to follow. Although it is not always possible, there is potential for the series of failure events to be cut short, and prevent a catastrophic failure down the line.

Rebound represents the behaviour undertaken in order to cut the series of sequencing failure events short, with the goals of preventing the catastrophic failure events from occurring and returning to a normal state.

For example, to prevent a free-falling plate from colliding to the floor and breaking, a safe approach with high probability and damage mitigation in mind would use any available tool to slow down the falling of the plate, to the point where the plate is not damaged, or just chipped imperceptibly. Returning to a normal state, while preventing further failures, would imply grabbing the plate from the floor, with awareness of the factors which made it fall in the first place.

Conditions for Rebound

For a scenario to represent a rebound problem, based on the definitions presented above, the following three elements are sufficient conditions that need to be satisfied by the proposed environment:

1. One **normal state**, with a generally clear optimisation goal, and a goal with an expected return with little variance;
2. At least one **failure** state, which puts all agents at risk of further failure events;
3. At least one **catastrophic failure** state, which needs to be avoided in the pursuit of returning to the normal state.

The ablation of either of these conditions would skew the problem towards robustness, damage mitigation or more generic damage recovery problems.

3.3 Deadlocks in MAS

3.3.1 Softening Deadlocks for Continuous MAS

The definition of a deadlock in a multi-threaded program is reasonably clear, since the acquisition of resources and their release are discrete actions in a discrete environment, with clear deterministic outcomes. However, in a continuous environment, even if the time or actions are discretised, a problem is introduced in the sense of the practicality of converting the definition directly from programs to continuous multi-agent systems.

There exist a couple of assumptions in open space sandbox environments which can make the direct translation complicated. Firstly, it can be rare that an environment contains explicit cases of locking/unlocking objects or other type of resources. Also, even if they exist, it can be the case that a functional deadlock still occurs. Secondly, a case can be made that an infinite amount of actions can still be taken by the agents in the environment, yet none of these actions directly benefits the co-operative or competitive goal of the agent, leading to a conceptual livelock abstraction. Finally, there is a new concept which can be introduced in a MAS, and that is how safe is a certain action or state. This can make certainly possible actions potentially unrealistic, making the baseline deadlock definition highly impractical in safety-aware scenarios [19].

A proposal would be to formalise symbolically the definition of a deadlock, in order to be easier to implement as a shield [85] or simply to design problems in such a way as to solve or prevent the occurrence of the deadlock condition. This definition, however, should be softened up as compared to its counterpart in multi-threaded programs, to circumvent the aforementioned assumptions which come into play in open multi-agent environments. In the following subsections, the term "agent" will be used instead of thread, since there will be a slow build-up from the formal thread deadlock definition to the more practical definition for multi-agent systems.

Weakened Deadlock Definition

A simple deadlock occurs when a set of agents $i \in N$ attempt to acquire mutually exclusive resources $r \in R$ which other agents hold, and will not release until they acquire all resources they need. Let $needs(i, r)$ represent the act of agent i trying to acquire resource r , and $has(i, r)$ representing the fact that agent i has exclusive ownership of resource r . Please note that, although it is possible for resources to be shared by multiple agents, in order to allow deadlocks to occur in a program/environment, all resources will be considered to have a limited amount of agents which can acquire them at the same time, and these resources will be split into sub-resources with a limit of one agent, for the sake of brevity.

The aforementioned intuition can be formally defined as:

$$\exists S \subseteq N. \forall i \in S. \exists r \in R. \left[needs(i, r) \wedge \exists j \in (S \setminus \{i\}) [has(j, r)] \right] \quad (3.4)$$

This definition, however, does not take into account the impracticality of modelling resources to own in multi-agent open environments. One way to circumvent this impracticality is the replacement of *ownership* of resources with the *action* of acquiring them. For instance, $needs(i, r)$ can be translated as $tries(i, lock(r))$, where $tries(i, a_i)$ represents the attempt of agent i to perform action $a_i \in A_i$, A_i is the action space of agent i and $lock(r)$ is the action of acquiring resource r . Similarly, the formula $has(i, r)$ can be converted into $blocks(i, lock(r))$, where $blocks(i, a)$ represents the fact that agent i blocks action $a \in \bigcup_{i \in N} A_i$ from being taken, through any means (i.e. cars cannot move in the space occupied by another car). For brevity, let us assume that the dynamics of action a consider the current state of the agent performing it.

Then, the redefinition of a deadlock in terms of actions taken, as opposed to resources owned, can be defined as:

$$\exists S \subseteq N. \forall i \in S. \exists a_i \in A_i. \left[tries(i, a_i) \wedge \exists j \in S. [blocks(j, a)] \right] \quad (3.5)$$

or

$$\exists S \subseteq N. \forall i \in S. \forall a_i \in A_i. \left[tries(i, a_i) \rightarrow \exists j \in S. [blocks(j, a)] \right], \quad (3.6)$$

where the former definition is closer to how a deterministic program thread would perform its actions, and the latter is closer to a cooperative or competitive agent, which could take a potentially infinite amount of actions at any given time. The latter, therefore, expresses a deadlock which does not allow the agents caught in it to perform any actions.

Note that, since the deadlock's occurrence is no longer constrained on the acquisition of resources by other agents, it is possible to have an agent deadlock itself by blocking its own movements (i.e. the game of Snake).

To weaken the requirements of the deadlock's existence more and allow more unique deadlock dynamics to occur, it is possible to remove the constraint of requiring a single agent to block the action of another agent. It is possible for certain actions to require multiple agents to stop it in unique situation (i.e. tackling), or for an agent to be deadlocked by no agent, but only by the environment's dynamics. Therefore, one of the weakest formalisations of the existence of a deadlock in a multi-agent system could be:

$$\exists S \subseteq N. \forall i \in S. \forall a_i \in A_i. \left[possible(a_i) \rightarrow \exists J \subseteq S. [block(J, a)] \right], \quad (3.7)$$

where the definition of $block(J, a)$ is an extension of $blocks(j, a)$, which now allow a subset of agents $J \subseteq N$ to block a certain action from occurring, and $possible(a)$ validates that action a can be taken, given the environmental constraints (i.e. excluding interference of other agents).

Practical Extensions

To introduce practicality for Continuous MAS, definition 3.7, there are two assumptions which were considered. Firstly, an agent will be able to take a potentially infinite amount of actions at any given time. However, many of these actions could not be considered

useful in regards to the goal the agent is trying to achieve. Secondly, safety can be a concept of high importance, which has also been of high priority in literature [19], with much focus being on shielding [85] or other formal methods of representing unsafe situations [111].

For the first assumption, definition 3.7 can be strengthened through the concept of preference and utility. With those, one could redefine the idea of possible actions with the idea of an agent *wanting* to take those actions. Therefore, through the definition of a $wants(i, a_i)$ formula, representing the idea that an agent attempts to take action a_i in order to achieve its goal with better odds, the deadlock condition can be redefined as:

$$\exists S \subseteq N. \forall i \in S. \forall a_i \in A_i. \left[wants(i, a_i) \rightarrow \exists J \subseteq S. [block(J, a)] \right] \quad (3.8)$$

For the second assumption, similarly, the definition of blocking actions can be redefined to consider the *probability* of an action to lead to an unsafe state (i.e. a crash). This extension should mostly require just a redefinition of the $block(J, a)$ formula to take in the probability that a certain action a will lead to an unsafe state due to the interference of agents $J \subseteq N$, and their possible actions based on their own policies $a_j \sim \pi(o_j)$. Formally:

$$block(J, a_i) \triangleq \mathbb{E}_{a_j \sim \pi(o_j), \forall j \in J} \left[crashed(i, t+1) | a_{it} = a_i, \forall i \in J \cup \{i\} \right] \leq \Gamma, \quad (3.9)$$

where $crashed(i, t) \in 0, 1$ is whether agent i crashed at time step t , action $a_{it} \in A_i$ is the action taken by agent i at time step t , and $\Gamma \in [0..1]$ is a real hyper-parameter constant, representing the threshold over which the probability of crashing is considered too high for the action to be taken. Otherwise, the definition stays the same as for 3.8.

3.3.2 Deadlocks in Traffic

Traffic scenarios represent an open environment, where a set number of cars (agents) attempt to reach a destination, given a starting location. Given the vastness of the environment, and the relative small amount of agents which can occupy a space by themselves, it is impossible to define a clear deadlock by definitions 3.4 to 3.7. However, given extensions 3.8 and 3.9, it is now possible to formalise what can be called a *soft deadlock*.

This way, we can define the formula $wants(i, a_i)$ as the clear preference of cars to move forwards (to utilise highest safe velocity) and towards their goal state (to take maneuvers which shorten the distance between its current state and goal state), and $block(J, a)$ as the idea that cars should not get too close to one another, or it could lead to an accident (i.e. less than 0.5m distance provides the potential for an accident). A diagram representing a deadlock based on this definition is in figure 3.2.

Deadlock Definitions Applied

To express the importance of the extended deadlock definitions 3.4-3.9 in their application in practical multi-agent systems, for each definition, an example of the possible expressed deadlocks have been provided in figures 3.3a to 3.4b.

Definition 3.4, highlighted in figure 3.3a, as mentioned above, is just the direct translation of the deadlock conditions into second order logic. This can work well in environments with discrete actions and states, but need certain adaptations if they are to be applied to environments with continuous dynamics. It is not impossible, since in much of the currently existing traffic management literature, a popular approach of modelling observation

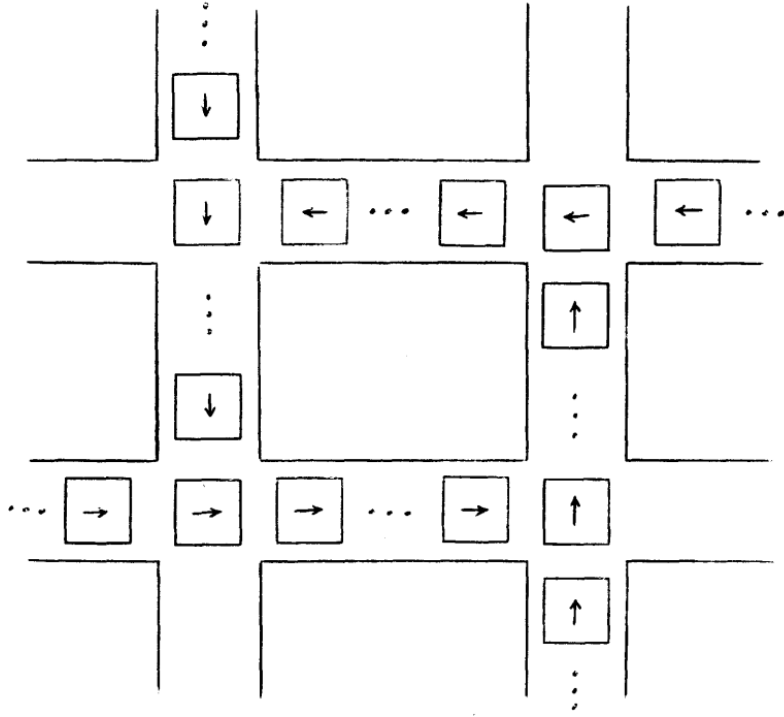


Figure 3.2: Intersection Deadlock from [35]

dynamics is the *occupation grid*, which works by abstracting out each lane into a grid-based representation, with the potential of giving the side length of one square of the grid a larger or smaller dimensionality, depending on the use case and the compute power which can be spared. In our example, to model the deadlocks, a natural way to do so is by modelling vehicles to try to acquire/reserve squares on the grid, which they want to occupy. If said squares are occupied, the agents wait until they are evicted. The downside of this approach is that grid occupancy essentially abstracts out the dynamics of the model to a discrete space, therefore disincentivising the exploitation of the continuous environment's finer grained dynamics. As in, by abstracting out the possibilities of an environment, it is likely for potentially more optimal, or even safer, solutions to be overlooked. Furthermore, this abstractisation requires high amounts of precision to ensure the correct grids are considered occupied, which is a factor that cannot be guaranteed in real life situations.

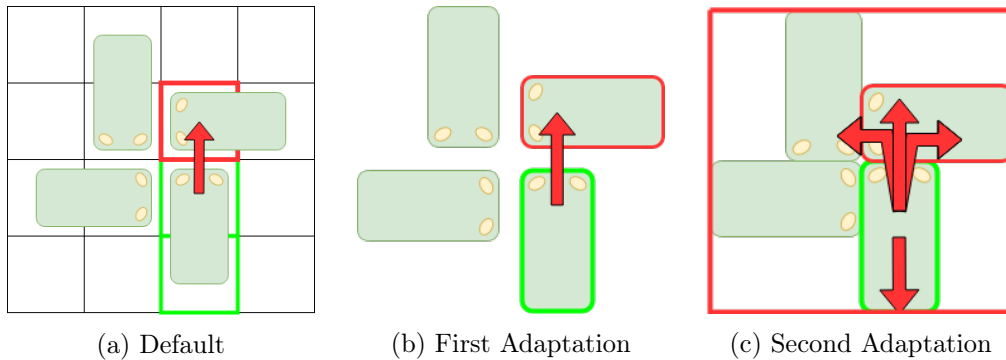


Figure 3.3: Deadlock Definitions applied to Deadlocks in Traffic

The first adaptations 3.5 and 3.6, highlighted in figure 3.3b, break the necessity to abstract resources by allowing the actions of the vehicles to act like resources, and the

blocking of said resources to be done by the collision of vehicles. This weakening of the deadlock condition already introduces a degree of practicality in the system, yet it still maintains two assumptions from the previous iteration: first, that there must be a single action that each agent considers it wants to take, and second, that each agent is provided a perfect model of the environment, and can pre-plan at least one action ahead, in such a way that it knows to stop and wait if the given action would lead to a collision.

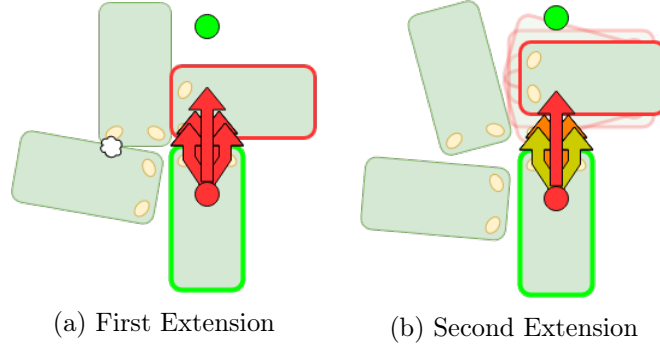


Figure 3.4: Deadlock Definition Extensions applied to Deadlocks in Traffic

The second adaptation 3.7 represents a non-practical weakening of the deadlock condition, which is highlighted in figure 3.3c. Since deadlocks occur now if agents are unable to take any possible action, all agents in the deadlock need to be constrained from every direction in order for this condition to occur. This definition, however, also emphasises strongly what is a full deadlock in a continuous MAS, and as mentioned earlier, represents a good starting point to provide relaxations on the deadlock's condition.

The first extension 3.8, highlighted in figure 3.4a, introduces the concept of desirable actions to be taken. This relaxation depends on every agent having a goal state (in the figure, this is a green dot for the vehicle highlighted with green borders), and prioritising actions which directly lead the agent closer to the goal. This relaxation drops the assumption of the single action being considered, while at the same time restricting possible, but not directly practical actions. This relaxation by itself does not yet drop the assumption that the agents are being given perfect models of the environment, in order to correctly predict the expected outcome of actions taken. Furthermore, it maintains the strengthening provided of formula 3.7, which might signify a more clear existence of a deadlock, but at the same time introduce unsafety in the environment, as deadlocks do not occur unless the agent cannot move forward anymore. That is, only if the agents are barely touching or have already crashed.

Finally, the last extension 3.9, highlighted in figure 3.4b, introduces the probability of crashing in the definition of blocked actions. This introduction maintains the possibility of detecting deadlocks when they do occur, while preventing crashing or touching when deadlocks happens. Furthermore, a degree of behaviour inference can be added to this definition, such that the probability of blocking takes in mind not just the current position of the blocking vehicle, but also its trajectory. Given knowledge of this vehicle's intentions, it can lead to a robust and practical methodology for deadlock detection. This definition finally drops the assumption that the agents require a perfect model of the environment, allowing this condition to be observed even in real life scenarios.

Deadlocks as a Rebound Problem

Deadlocks in traffic are representative of a rebound problem, since they adhere to the sufficient definition of rebound. The scenario's normal state represents fluid traffic flow, which provides all agents fair opportunity to traverse the given cross intersection. The failure event is the deadlock itself, which not only blocks the movement of at least 4 agents, but also blocks all incoming traffic arteries. The catastrophic failure occurs if any of the cars crash during their attempt to resolve the deadlock.

3.4 Simulated Environments

In the scope of this dissertation, there are two types of traffic intersections which have been developed and have been taken into consideration for evaluation: the simple cross intersection 3.5a, and the four arm junction 3.5b. In this section, a description of the layout of the internal representation of each environment will be provided, as well as motivation behind the choices which were taken in their design.

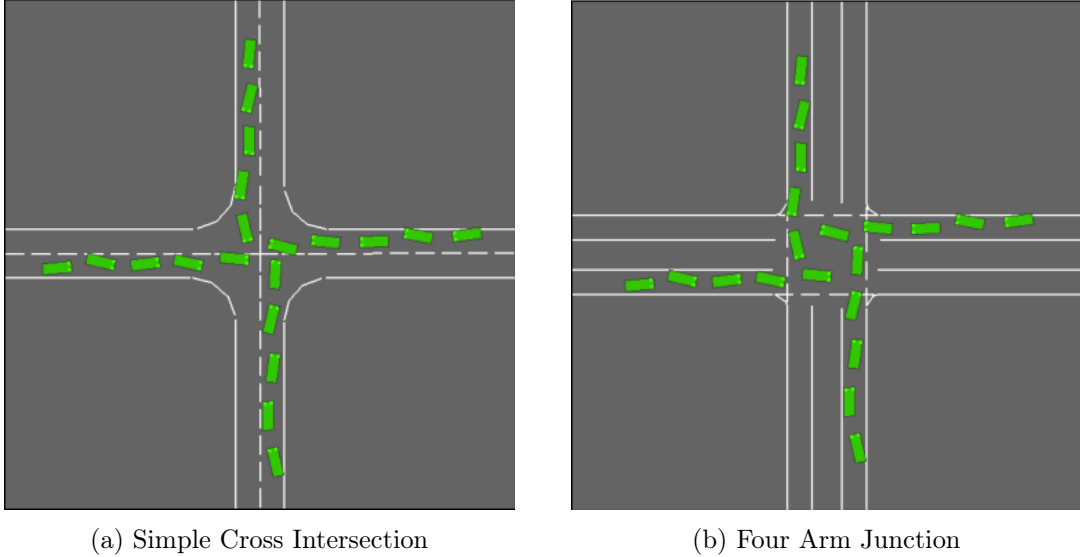


Figure 3.5: Simulated Traffic Environments with Deadlocks

3.4.1 Deadlocked Starting State

The vehicles in any of the two environments can be initialised at training time in two ways, either in a standard 4 way intersection position, with even utilisation of every arm, or in a deadlocked state, as in figures at 3.5. Each agent is placed after a minimum offset behind the agent in front of it, with random deviations to their lateral, longitudinal and angular deviation. Randomised states at initialisation are used for two reasons: firstly, to introduce more robustness in the behaviour of the agents, by putting each agent in a different state compared to their last iteration, and secondly, to allow the deadlock in the intersection to be solvable. The scenario that the second reason attempts to avoid is when all agents start in the same relative position, at the same offset, from each lane in the environment. Since the agents are homogenous and share state definition and policy, if each agent is faced with the same relative state, they could end up in a situation similar to a livelock: going back and forth, with the same accelerations, at the same time.

This situation, albeit rare, is not impossible, especially in environments with less fine grained knowledge of vehicles' states, and needs to approximate coordinates. A noteworthy

observation is that this problem is similar to a family of problems in robot control theory called *singularities* [112], and shares a practical and sufficient solution: introducing a small amount of random perturbation in the input. Since the random noise is small, and the action space is simplified, the difference is imperceptible in terms of the accuracy of the policy based on the input, but it dramatically minimises the chances of a singularity scenario to occur by providing a slight difference which the policy of each vehicle can exploit, and choose different actions.

3.4.2 Internal Representation

A choice made for the purpose of simplifying the expected behaviour in the experimental demonstrations for this dissertation was to use an action space called *DiscreteMetaActions*, further explained in 3.4.3. This action space meant that the path was pre-determined, and the vehicles would follow it using a very simplistic PID controller. This allowed the autonomous agents to focus on controlling their acceleration, as opposed to their full range of dynamics.

In order to permit behaviour that could deal with the occurrence of a deadlock in an intersection, the use of *DiscreteMetaActions* required the definition of the environment with the vehicle's pathing in mind. This pathing is represented internally by the Highway-Env simulator as a graph, with the vertices being the lines followed by the agents. The layouts of the routes which an agent can follow are highlighted in figure 3.6.

These routes have been designed for the purpose of deadlock solving, and have the added benefit of working as a form of dynamics limitation, to speed up the convergence to an optimal deadlock resolution policy. When an agent on one of the paths tries to move backwards, if it moves out of the lanelet (that is, on or behind the node behind it), the agent will not be able to make any more backwards movements. This decision was made with the design in mind to limit backwards movement up to the point where it becomes unnecessary for a vehicle to keep moving backwards. For instance, a vehicle in the middle of the four arm junction, between any two yellow nodes, would not need to move further back its lane, but only to stay in between the separated lanes. Similarly, a vehicle deadlocked in the cross intersection does not need to move further back than the red node, in order to allow other vehicles to pass in front of it.

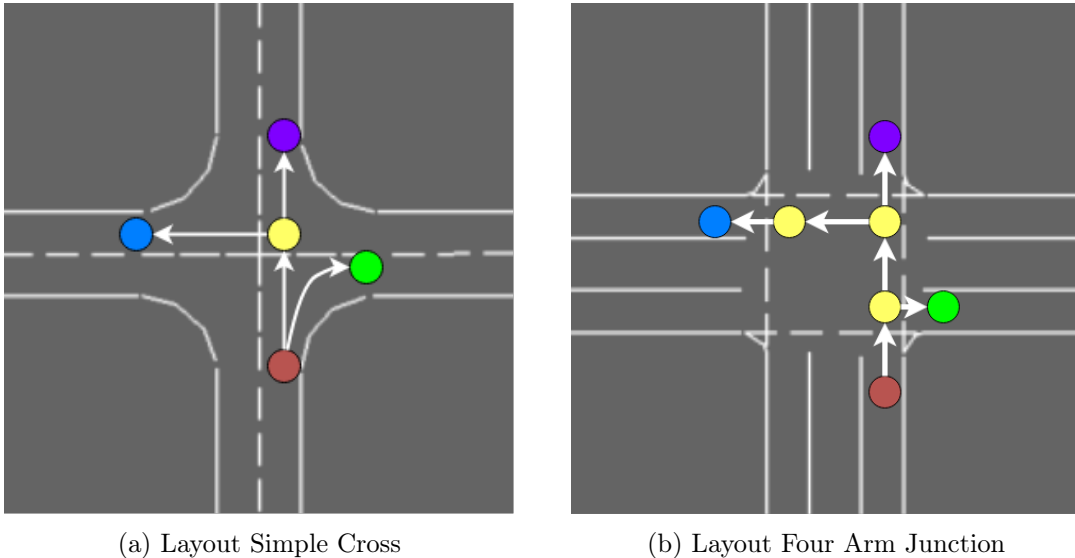


Figure 3.6: Route Node Layout

3.4.3 Vehicle Dynamics

In the following subsections, a description of the state, observation and action spaces of the vehicles will be provided.

Observation Space

The primary observation space for the agents is a grid of information observed from each agent in the environment. The agent is trained with knowledge of the following features from each agent:

Title	Description
x, y	the coordinates of the ego vehicle/offset coordinates to the observed vehicle
vx, vy	velocity of the vehicle
$\cos_h, \sin_h$	current heading of the vehicle (i.e. movement direction)
$\cos_d, \sin_d$	heading to the destination of the vehicle

The set of $\langle x, y \rangle$ coordinates allows the ego agent to reason about its own location, as well as the location of the rest of the agents. The set of velocities $\langle vx, vy \rangle$ allows the ego agent to extrapolate time differential information about how fast other agents are moving, without the need of an RNN or previous frame information. The trigonometric angle of the heading of each vehicle $\langle \cos_h, \sin_h \rangle$ provide information on how close should an agent get to another agent, given its angle. And finally, the angle to the destination $\langle \cos_d, \sin_d \rangle$ provides information in terms of the destination which every agent attempts to reach, providing data that can be used for decision-making in the intersection: which agent moves first.

State Space

Since the [MADDPG](#) algorithm variations implemented make use of a centralised critic, it is possible to leverage the convenience of the [CTDE](#) training framework. The simple approach to model the critic would have been to concatenate the observations of each agent during one step, and return the approximate Q value of the set of observations. Since each agent’s observation introduces the relative observations it has of the other agents nearby, this meant that the input size would have scaled by a factor of N^2 for each new agent in the environment (up to the hard limit of each agent’s perception), significantly reducing the potential of the critic for scaling. Furthermore, it can be argued that the observation of an ego agent of its relative offset to any other agent can be regarded as redundant information, since it is information that can be arithmetically inferred from the ego states of any two agents.

The approach taken for the critic’s state space input is the extraction of the ego state from the observation of each agent; that is, only the information regarding the vehicle making the observation. Intuitively, this is sufficient input for the centralised critic network, since the value of a state at any particular point in time should not require the local observations of an agent to be computed. This state maintains the elements discussed in the observation space.

In order to not introduce biases for any particular aspect of the dynamics of the environment, further preprocessing of the state information is avoided.

Action Space

The primary action space for the agents is a set of discrete actions called "DiscreteMetaActions", represented by an enum of 3 elements and a set of target speeds.

Title	Description
<i>IDLE</i>	maintain current velocity
<i>SLOWER</i>	move target velocity lower by one index
<i>FASTER</i>	move target velocity higher

The set of target speeds can be configured to various degrees by the designer of the training environment. Currently, it is set as $[-3, 0, 4.5, 9]$, representing a slower speed for moving backwards, the target speed for stopping, the target speed for moving cautiously, and the highest possible speed, for highest throughput.

Reward

For this particular problem, the reward is modelled with cooperation and fairness between the agents in mind. For cooperation, the reward is shared between the agents, and for fairness, it represents an average between the individual reward of each agent. Formally, the reward of all agents has the following equation:

$$r_i(t) = \frac{1}{N} \sum_{i \in N} \left(r_c(t) \text{ or } r_g(t) \text{ or } (v_i(t) - b_v) \right), \forall i \in N$$

where $r_c(t)$ is a negative penalty, given if the car is crashed at time t , $r_g(t)$ is a positive incentive, when the car's goal destination is reached at time t , which in this case is out of the intersection, and finally $v_i(t)$ and b_v represent the velocity of agent i at time t , and a bias velocity respectively, incentivising the agents to move forwards. The reward of each agent is given in the priority showcased in the equation: if the car is crashed, just the crashed penalty will be given. If it is not crashed, the positive incentive for reaching the goal is given. Otherwise, higher speeds are rewarded.

The first two rewards being potentially given, r_c and r_g , allow the agents to prioritise the safe passage through the intersection of all agents in the traffic scenario over the quickness of its resolution, while the third incentivises exploration through forward movement. Furthermore, the averaging of the reward ensures that no agent is left behind, and the resolution of the scenario does not lead to the starvation of any lane.

3.5 Algorithms Implemented

In the following section, a description of the algorithms implemented for this dissertation will be provided, as well as motivation behind their different variations.

3.5.1 DQN

The Deep Q Network from RL-Agents [31] was used, without further modifications, on the environments proposed in this thesis, as a baseline to improve upon by other algorithms, and to compare with a CTDE multi-agent approach. Since DQN by itself is an algorithm that, by design, cannot be trivially adapted for CTDE, it was used in an Independent DQN for MARL setting.

The algorithm trains a single Q policy network, which is shared by each agent in the traffic environment. It takes in, as observation, the vehicle’s dynamics, as well as the dynamics of up to 15 neighbouring vehicles, and returns the softmax probabilities of the actions with the highest potential value returned, with the definition of the observation and action space being explained in section 3.4.3.

For training stabilisation, the DQN uses a standard experience replay buffer, which is shared by all agents in the environment, as well as a standard target Q-network architecture, on time-delayed training. This replay buffer stores tuples of the form $\langle o_t, a_t, r_t, o_{t+1} \rangle$, where o_t , a_t , o_{t+1} are the observation, action and next observation of a single agent in the environment at time t , and r_t is the shared reward of all agents in the environment. Since the observation space is relative to each agent, no matter from which direction they are coming from, it means that each agent can learn from the experience of the other agents in the environment, based on their own observations.

The problems in this case would be, firstly, the non-stationarity in time, introduced by the lack of knowledge of the actions of the other agents in order to reach o_{t+1} , and secondly, the usage of a shared reward, which makes it difficult to assign credit to any particular agent in the system, and introduces non-linearity in the approximation of the true value function of the environment.

Ego Attention

An additional feature of the given DQN in the library was the introduction of social attention as a mechanism for intersection traversal, in environments with social, but ultimately non-learning agents [75]. The mechanism works on the observation of the environment, and compares the kinematics of the ego vehicle to the kinematics of the other vehicles that can be observed in the environment. Intuitively, this makes the agent give more importance to observations of some agents, compared to others. Formally, this particular attention mechanism uses a linear encoding of the ego kinematics observation as the query $Q = f(o_i[0])$, and linear encodings of the kinematics of all observed vehicles represent the set of keys and values, $K = g(o_i)$ and $V = h(o_i)$. How Q, K and V are used is explained in the background chapter, in section 2.4.2. By deploying multiple heads of attention, each head can focus on a subset of different vehicles.

For the rest of the thesis, social-attention and ego-attention will be used synonymously.

3.5.2 MADDPG

One of the contributions of this dissertation was the implementation of a MADDPG algorithm as part of the RL-Agents library. This algorithm is fully customisable to work on any potential redefinition of the observation, state and action space, discrete or continuous. Furthermore, on top of the standard baseline definition of MADDPG, two more variants were developed to use attention mechanisms, with simple *self-attention* and the better performing *ego-attention*.

A CTDE version of MARL was developed for this project in order to experiment with an alternative solution to the independent training DQN algorithm. CTDE architectures has, formally, better convergence guarantees for multi-agent scenarios, leading to higher levels of coordination, since the critic should return more accurate estimates of the value of the environment, particularly given the shared reward defined in the environment. Furthermore, the new architecture allows the training of agents with continuous action spaces

in multi-agent environments, which was not possible to do before in Highway-Env. Prior to this work, StableBaselines3 [107] was the only [RL](#) library with continuous action space compatibility with Highway-Env, but it only worked in single agent scenarios, like parking.

Baseline

The [MADDPG](#) algorithm implemented for RL-Agents [31] was inspired in design by the one proposed by OpenAI [56], referenced in Appendix A as a variant of algorithm 3, with further modifications based on the non-deterministic Policy Gradient for the usage of discrete actions. As in, the Actor network, instead of returning the continuous value for the continuous action, a set of N values are returned through a softmax activation layer, each value in the set representing the probability of a discrete action in the set of N possible actions that can be taken.

Both the actor and the critic, in the baseline version, are [MLPs](#) of variable sizes. The networks of the actor and the critic are not shared in any place, as one would usually do for actor-critic algorithms, since the sizes of the input layer of both networks differ significantly, and are different in scope. The input of the actor is the observation of each agent, as described above and in section 3.4.3, whereas the critic takes in the state of the environment instead, as described in 3.4.3. This can pose a challenge for the learning algorithm, since the actor and the critic are trained on different inputs, and will need to train in parallel in a way that ensures they both converge to the optimal policy and value function respectively.

A modification to the algorithm from the base version was related to the usage of homogenous actors and critics and, consequently, the training of only one actor and one critic for each agent in the environment. The choice to undertake this option was to enhance the scalability of the learning algorithm, and allow the training of a variable amount of controllable autonomous agents, for both the current work and any further research into multi-agent autonomous vehicle control done on Highway-Env [29]. To facilitate this choice in architecture, the logic of the input of the critic had to be modified, since the insertion of the same state for each agent would yield identical results for the value function, even in situations where the reward is not shared, which is incorrect behaviour. So, the definition of the Q function has been slightly altered as $Q(s_i, s_{\setminus i}, a_i, a_{\setminus i}; \theta^Q)$, where the state of the environment and the collective actions taken by all agents are split into the state and action of a single agent, and the other states and actions. With this modification, for every agent, the Q function will return the correct value function of that agent.

To stabilise the training, the replay buffer now stores tuples of the states and actions of all agents $\langle \mathbf{x}, a, r, \mathbf{x}' \rangle$. Furthermore, instead of the standard target network update every X time steps, the soft-update of the target networks is employed, as used by both the original [DDPG](#) [38] and its multi-agent counterpart [56]. Both the actor and the critic need target networks in this version of the algorithm.

Ego-Attention

Inspired by the social attention mechanism [75] used in the [DQN](#), and by the way attention is used in MAAC to improve the scalability of the critic [62], this thesis proposes an extension of the [MADDPG](#) with social attention, where both the actor and the critic are enhanced with an ego attention mechanism. The actor network works the same as the [DQN](#) in terms of how the observation of each agent is passed through the input layer. The critic had to be adapted to the use of the state as an input instead, where a linear encoding of the ego state becomes the query $Q = f(s_i)$, and linear encodings of the state

of all observed vehicles become the set of keys and values, $K = g(s)$ and $V = h(s)$.

Self-Attention Extension

On the same line of work, another contribution proposed in this dissertation, still inspired by the Leurent et. al.'s work on social-attention [75], is the usage of self-attention on top of the social-attention mechanism. After the embedding of the ego and other observations as query and keys, Q and K are concatenated and passed through a layer of self-attention, before being split and processed through the aforementioned social-attention mechanism.

This variant is introduced mostly to observe whether there are any more useful features that can be introduced into the initial embeddings through one more layer of attention.

Chapter 4

Evaluation

In the following chapter, the methods used to measure and validate the performance of the agents in the deadlocked intersection environment will be showcased. Given the methods, the results will be presented and discussed, as well as the challenges that have been met during the experimentation part of this dissertation.

Nominally, this chapter will discuss:

- The setup of the environments and algorithms used during the experiments.
- The training procedure of every algorithm variation, as well as their performance validated on the different environments.
- The training performance of every algorithm variation, discussed in context with their average performance on the validation scenarios.

Video recordings with highlights of the trained behaviour are posted on YouTube¹.

4.1 Experimental Setup

The primary goal of the agents being trained is to coordinate in a way that ensures that, when they get caught in a deadlock in an intersection, the agents are able to safely unlock themselves and restore the environment to a natural flow.

The starting point of either environment is an intersection occupied in a deadlocked cycle by four agents, one agent for each incoming lane. An experiment is considered a success if at least half the agents are able to reach the goal in the environment in the time available, and no collisions occurred.

4.1.1 Environments

There has been instrumentation with two environments in two variations. The layout of the intersection can be either the simple cross, marked as *Cross X*, or the four arm junction, marked as *Junction #*. The placement of the vehicles in the intersection can be either simple, marked as *S*, leaving the vehicles at a randomised offset from the center of the intersection, or they can be deadlocked, marked as *D*, where the agents are placed a priori in a deadlocked state within the center of the intersection. Further information on the lower level configurations of the environments can be found in Methodology, in section 3.4.

¹https://youtube.com/playlist?list=PLHbOnARTMx6vkQnsywIykHPcT_DI4k8GD

The reason training and validation has been done on a *Simple* version of the intersection was threefold: i) to showcase the performance of the implemented algorithms in pure traffic intersection traversal, trained as a damage prevention scenario, ii) to showcase the performance of algorithms trained primarily on deadlock prevention when reaching a deadlocked state, and iii), to compare the previous algorithms with the performance of variations trained directly on the deadlock resolution.

4.1.2 Algorithms

There were two main frameworks of algorithms which were implemented, both of which have been extended to use attention mechanism as their alternative approach, with various amounts of attention heads.

The main frameworks used were [DQN](#) and [MADDPG](#), both explained in detail in Methodology, in section 3.5. The hyperparameters chosen for each framework have been provided in Appendix B.

4.2 Evaluation Results

In order to compare both of these frameworks, as well as the impact the attention mechanism variations have, a version of each algorithm was trained for each environment configuration. The performance of the episodes will be measured based on their success rate in freeing up the intersection over 10000 episodes.

The [DQNs](#) were trained for 50000 episodes, while the [MADDPGs](#) were trained for less than 25000. The reward function used was discussed in the Methodology in section 3.4.3, and was hyperparametrised as in table B.2. Other hyperparameters related to the training can be found in Appendix B.

Algorithm Variant				Cross X			Junction #		
				S $\rightarrow$ S	D $\rightarrow$ D	S $\rightarrow$ D	S $\rightarrow$ S	D $\rightarrow$ D	S $\rightarrow$ D
DQN	Baseline			44.72%	13.53%	8.3%	68.95%	36.95%	2.4%
	Attention Ego		1h	63.76%	44.32%	11.15%	75.15%	76.27%	0.7%
			2h	87.18%	47.85%	7.7%	84.80%	86.90%	1.75%
			4h	84.97%	44.01%	16.25%	86.84%	74.92%	1.15%
	Baseline			49.49%	6.9%	0.2%	48.91%	63.69%	41.05%
MADDPG	Attention	Ego	1h	42.02%	0%	4.65%	60.86%	39.05%	16.95%
			2h	48.27%	0.30%	2.9%	57.57%	96.49%	0.4%
			4h	51.94%	0%	7.6%	58.60%	26.36%	17.2%
			1h	0%	0.01%	0%	55.52%	95.77%	97.75%
		Self	2h	0%	0%	0%	64.97%	76.01%	26.15%
			4h	17.82%	0%	0%	62.99%	71.76%	85.25%

Table 4.1: Performance of RL agents on Deadlock Resolution

The results of the validation of each trained algorithm can be found in table 4.1. The symbol $A \rightarrow B$ represents the training of a given algorithm configuration on scenario variation A , and the validation of the same algorithm on scenario B , where $A, B \in \{S, D\}$, where S and D are the simple, and deadlocked scenarios respectively. Note that, for all variants, the validation of deadlock-trained algorithms on simple intersections $D \rightarrow S$

has been omitted, due to the high probability of deadlock-trained algorithms to not have explored the starting states of simple intersections, leading to an unfair comparison.

4.2.1 Discussion

The results of the training highlight the difficulty of the resolution of deadlocks in traffic intersections, where the best success rate in cross intersections is less than 50%, and that only for agents which are trained a priori on the deadlock. This must occur due to the high probability of the movement of vehicles to lead to a collision, due to their general proximity to one another, the non-linearity of the observation and action spaces, as well as incomplete control over the dynamics of the vehicle (the agents follow a pre-defined route through [PID](#) control).

Attention mechanisms generally improve the success rate of the majority of scenarios compared to the baseline [MLP](#) model, with a positive performance difference of over 30% between same framework algorithms tackling the same training/validation split. In more detail, the performance improvement of ego attention [DQN](#) algorithms on $D \rightarrow D$ scenarios is 40% over the baseline on average, which amounts to a relative improvement of 3.5 and 2.3 times for cross intersections and four way junctions respectively. For the [MADDPG](#) framework on four way junctions, the total performance reached with attention mechanism, for multiple variations, is over 95%. This showcases attention as an extremely important aspect when it comes to resolving scenarios requiring high levels of coordination.

The variations of the [MADDPG](#) framework, on the four way junction environment, are expected to have learned to exploit an imperfection in the deadlocked setup, which allows the scenario to be resolved for four agents without the need to move backwards. By synchronising their movement, the agents can move forwards at the same time without colliding into each other. Interesting to note that the [DQN](#) algorithms might not have discovered this exploitation, since the performance on four way junctions is still behind [MADDPG](#) in this scenario. This highlights a tendency of the [MADDPG](#) policies to be more precise compared to the [DQN](#) counterparts.

However, even with attention, deadlock resolution in the cross intersection seems to be particularly difficult to learn for the [MADDPG](#) framework, with only the baseline [MLP](#) network able to have some success, albeit drastically behind [DQN](#). In theory, this is a surprising result, since [CTDE](#) algorithms benefit from formal convergence properties which independent [DQN](#) does not have in multi-agent scenarios. However, as previously noted in the literature [103, 62], [MADDPG](#) has a tendency to be unstable during training, requiring more fine-tuned hyperparameters and/or more stabilisation methods in use. In this case, it is expected that [MADDPG](#) converged to a local optimum, choosing to make the cars move backwards perpetually instead of having the agents return to resolve the deadlock.

4.3 Algorithm Training Performance

The plots in figure 4.1 are provided for an understanding of the convergence rate of all agents, as well as

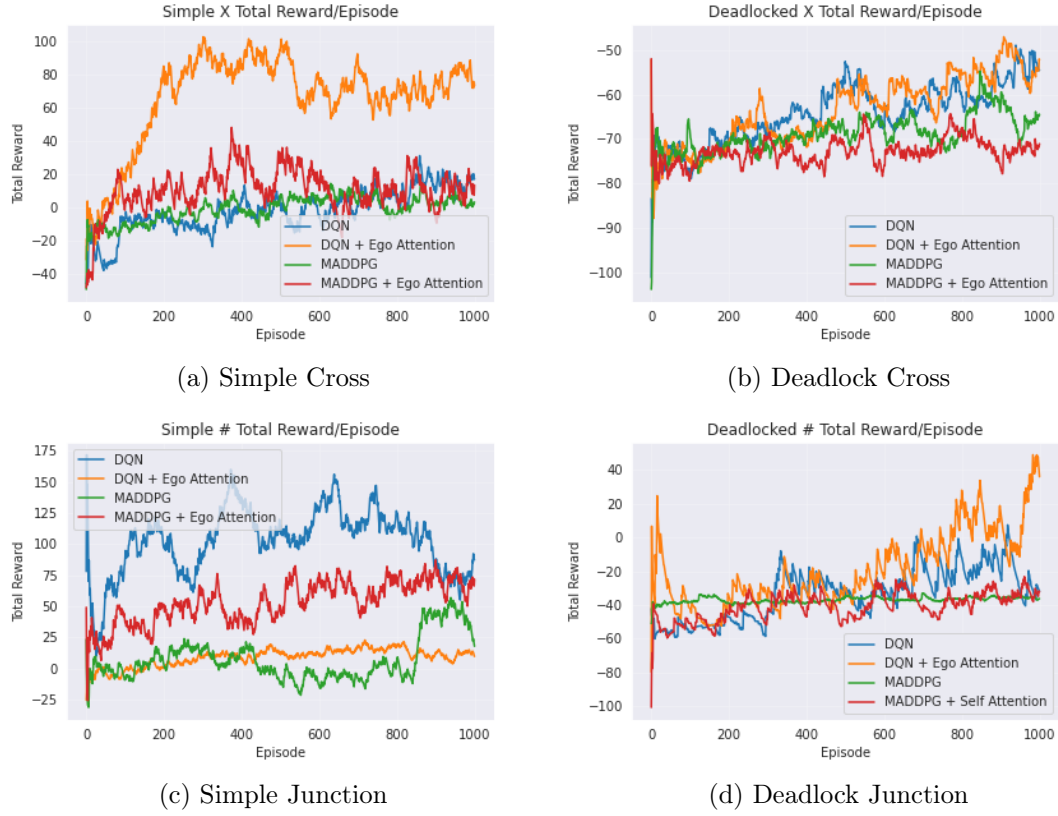


Figure 4.1: Performance of Trained Agents

4.3.1 Discussion

4.4 Key Takeaways

As the results in table suggest, the [DQN](#) on discrete actions returns highly competitive results, in many circumstances being better tuned to safely solve the deadlock compared to the agents trained with [MADDPG](#). This fact most probably transpires from the simplified network architecture which the [DQN](#) employs, which uses the same network for both value estimation and policy execution, as compared to the [MADDPG](#), which needs to balance the training of two networks in parallel.

Worth noting that, compared to [DQN](#) who runs for almost 50000 episodes in order to reach its peak performance, the [MADDPG](#) methods reach comparable performance in less than 25000, on average reaching their peak between 5000 and 10000 episodes. As generally acknowledge in the literature [58, 103], the baseline [MADDPG](#) suffers from is over-reliance on well tuned hyperparameters, which usually leads to unstable policies during training in the cases where they have not been tuned well enough.

Attention mechanism proved to be an important factor in the performance of the agents, with improvements of even 50% between the baseline MLP model and the best attention variant of the same learning algorithm framework ([DQN](#) or [MADDPG](#)). This is an expected result, since the mechanism should provide higher importance on features of significance from the agent’s observation, and similarly for the estimation of the value function of the simulated environment.

Overall, the problem definition, even for only four deadlocked agents in a simplified envi-

ronment, proves to be a challenging task for the baseline reinforcement learning literature on multi-agent reinforcement learning, with the potential of reaching highly performant policies in the right conditions.

4.5 Challenges and Future Work

The scenarios evaluated and results extracted in the current state of this thesis are in a smaller amount and, consequently, less conclusive than what was originally planned. This was caused by a series of shifts in the focus of the dissertation. Much time was spent familiarising with the state of the art, in terms of multi-agent reinforcement learning, traffic control, but also sim2real capabilities, since there was the plan of validating the behaviour in real life robotics. However, as it was discovered and discussed in the Methodology in section 3.1.1, there were considerations which did not allow the usage of the simulator that provided real life robotics validation. At that point, a significant portion of work done for the dissertation became obsolete, and the focus was moved to simulator-only.

The shift in focus, as well as the search for a valid backend that allowed the scope of the dissertation to be maintained, drastically reduced the amount of time that could be spent on instrumentation with variable hyperparameters and algorithm configurations. A single DQN experiment would take over 16 hours on a GPU to stabilise the policy to a satisfactory level, MADDPG experiments would take over 8 hours to catch up, and the hardware access was limited to at most 2 high-end GPU and 2 CPU machines. There were over 50 experiments that were run to completion and discussed in this evaluation, amounting to an estimated 700 hours of high throughput computations on 2-3 machines in parallel, on average. At least 300 hours more of compute time were spent on the training of experiments which were not discussed in the report, since they were relatively inconclusive. For instance, for scenarios with 8+ agents, no tested policy was able to converge, suggesting a need for AVs to either have a more stable traffic behaviour trained into them a priori, and/or other elements considered in the observation space, like Lidar or distance sensing, to make the target goal more linear. On top of that, sometimes experiments had to be redone, due to hyperparameter instrumentation, and the occasional situation where algorithms would not converge satisfactory the first time. Validation of performance also required at least one hour of compute time for every experiment, for a fair percentage estimation.

Furthermore, since the current state of the simulator did not have the mechanism in place to aggregate and measure the throughput of each controlled agent accurately, this element of the evaluation needed to be skipped in the current version of the report. This particular aspect is highly encouraged to be pursued in future work, due to the importance on leveraging the coordination capabilities of connected AVs not just for safety, but also for a faster deadlock resolution, fair for all agents.

Further work could be done to ensure a better estimate of the performance of the proposed algorithms, like training the same algorithm multiple times and returning the average performance of the best solutions in every configuration. However, an arguably preferred development should be the implementation of learning stabilisation features, like prioritised replay buffers, to improve the convergence speed, training reliability and, consequently, improve the performance of the current models, particularly the MADDPG model.

Furthermore, to capitalise on the ability of deterministic models like MADDPG to tackle continuous action spaces, future work could experiment with the performance of the

scenario with finer grained control on the acceleration of the agents. Even though continuous acceleration control will add more complexity to the dynamics of the environment, it will also make the expected behaviour more linear, and less dependent on the awareness of past states, making the training of [DDPG](#)-based policies, at the very least, more stable. Once this stability goal is reached, future work is highly encouraged to train and validate multi-agent policies on a larger scale deadlock, with more agents and/or more lanes.

Chapter 5

Ethical Issues

In the following chapter, the potential ethical issues which this work proposed are considered and discussed.

5.1 Protection of Personal Data

A continuation of this project, if deployed in a real life traffic scenario, will not require access to any driver's personal information, other than the current spatial locality of their vehicle, while in direct proximity with a vehicle's sensors and/or Vehicle-to-Vehicle (V2V) sharing capabilities. This information will primarily be used as observation knowledge, and potentially to infer the vehicle's intended route, but should never be stored for later use. As in, vehicles should be agnostic to the identity of the vehicles it perceives, and should not be able to recognise the same vehicle on a second meeting.

The only reason a self-coordinating fleet of vehicles could use the tracking of the human driver would be to predict its most probable traffic trajectory. Through the meeting of multiple agents in the city in a short time, the driver's trajectory could be deduced, and used in the rerouting of the cooperative agents to avoid congestions in that direction. This scenario is out of scope for this project, as the full path taken by the human agent is unnecessary to reach the deadlock resolution goal of the environment.

5.2 Misuse

5.2.1 Privacy

To continue the discussion above, especially in the case where the current project wants to allow coordination with non-communicating traffic agents, there will need to be a degree of prediction of the other agents' behaviour. This prediction, in theory, should be locally used and never stored.

However, since the project is intended to become widely available (under the BSD license), it is possible for people to use the principles designed in the project, and add on top of them methods of tracking the human drivers met in traffic. Through very advanced vehicle-mounted apparatus and algorithms, it is also possible to correlate the people met in traffic with their social media profiles. In the best case scenario, this information can be used to more accurately predict the behaviour of each driver in traffic. Otherwise, the usage can range from targeted advertisement to national or foreign agency surveillance.

Other than by modifying the license to specify legal repercussion, there is no trivial way to protect against this form of misuse.

5.2.2 Safety and Relaxations over Rebound Condition

The definition of the deadlock for [MAS](#), as well as the extensions which were in the subject of this dissertation, were proposed as practical alternatives to the strict conditions of the deadlock in early concurrent processing theory. Their primary goal was to allow the usage of the definition in environments which represent, conceptually, a deadlock, in the hopes of aggregating theory from more sources into the research of their safe resolution, like scheduling or game theory.

Unfortunately, it might be the case that such relaxations could be used maliciously. An example of such a use case assumes a situation where there exist regulations that require the deployment of agents in a public area only if they satisfy certain safety conditions, like rebound from failure. A person in charge of this legislation, either through ignorance or with malicious intent, inspired by this work, could permit relaxations of the safety conditions, for the sake of allowing more businesses to deploy their agents in such environments, and therefore influencing the economy for one's benefit. An argument can be made for such an approach, that since the environment in real life is continuous, there needs to be a degree of relaxation introduced in safety conditions, including rebound, for the sake of practicality. On the other hand, there is the question of where, and whether, there is a clear hard cap for these relaxations. Too much relaxation in risk-bound situations could lead to damaging results.

The solution needs to take into consideration both who is and who can be responsible for proposing the limitations on the agents entering public places, as well as permitted relaxation to such constraints. Every situation will require a different solution, and the people responsible need not only to have expertise in the domain, but also a level of professional noninvolvement with the parties directly benefiting from relaxed conditions. Without this minimal separation between the legislator and the beneficiary, a degree of bias could slip, and wrong decisions could be made for profit, which in turn could lead to the endangerment of all participants, innocent or otherwise.

Whether and how much should the professional team of legislators be compensated or penalised for the performance of agents which adhere to their conditions is still a philosophical problem under the same mantel as the infamous Trolley Problem [\[113, 114\]](#). A balance between safety and practicality should be considered, and the consequences should be owned, but in a way that is fair for every involved party. One cannot avoid all accidents 100% of the time, especially in social scenarios, and it is unfair to ask legislators to model safety restrictions which do not permit accidents from occurring, due to their inherent nature of occurring unexpectedly. On the other hand, the entire purpose of the safety legislation is to ensure a high degree of safe conduct.

On the rebound from failure safety conditions, since their main purpose is to ensure that an agent, when observing a failure situation, will do everything in its power to return to a safe state and prevent critical failures, a very strong point needs to be made for the relaxation of one of the environment's conditions to initiate a rebound. It is not impossible, and exceptions should be allowed for heterogenous environments (as in, not all agents need to adhere to the same degree of strictness to the rules, i.e. a car and a pedestrian), but such relaxations need to be limited by safety principles, taken in good faith and transparent to all participants of the environment where these agents are being deployed.

5.3 Workplace Disruption

Arguably, the main users of this technology will be the mobility-on-demand providers, like Uber, Bolt and other taxi companies, as they have the resources to deploy it first, once the governmental regulations are in order. From that point, once the technology gains traction, it could lead to a steady decrease in the demand to these companies for human taxi drivers.

There will still be a market for human drivers, as some people will stay reticent regarding autonomous driving vehicles, or could simply prefer the human company. After all, automated checkout stations have existed for a long time in supermarkets, but a large amount of people still prefer the service of a cashier [115]. Although, in theory, one could roughly estimate that the amount of paid human drivers will diminish, the highly skilled professional drivers should become better valued, with higher wages, potentially leading to higher levels of satisfaction for both worker and customer.

Chapter 6

Future Work

In this chapter, a set of potential future research goals are proposed, as a natural continuation of the contributions developed in the current work.

6.1 Rebound-Focused Reinforcement Learning

The definition of the newly proposed area of focus for learning-based approaches, namely Rebound from Failure, should be further studied and validated in relation to the new directions in which safe reinforcement learning is headed [111]. The definition of both Rebound and Deadlocks in MAS are highly correlated with the work undertaken to solve deadlocks in traffic intersections, and was only briefly discussed in relation to other safety-aware environments. Therefore, further research into how the current concepts fit in alternative practical use cases would help in fleshing out the field further, to understand its implications, and hopefully develop a general framework that can help pin-point the situations that agents need to be prepared for, and ensure they are covered during training time.

6.2 Deadlocks in MAS

A formal definition was proposed for deadlocks in continuous MAS, in order to make it easier to provide formal guarantees when tackling practical scenarios which could reach the point of a deadlock. One of the main inspirations for this objective was shielding [85], a concept used in Safe Reinforcement Learning to pinpoint and prevent the occurrence of failing states during training and execution. As mentioned above, the definition was proposed with traffic deadlocks in mind, but in theory, it should be general enough to be applicable to more multi-agent scenarios.

Therefore, work on deadlock avoidance and resolution which was previously done in discretised environments [83] could, with the help of this definition, be converted to environments with continuous dynamics, with the hopes of discovering emergent behaviour able to capitalise on and exploit the dynamics for policies which are more performant, and safer. This should be a natural step towards the deployment of policies from simulated environments, where potential deadlocks could occur, into the real world.

6.3 Realistic Simulations & Sim2Real Transfer

One of the initial objectives of this dissertation was, after solving the deadlock in intersections problem, to validate the emergent behaviour on the DuckieTown real life environment

[99]. Unfortunately, at the time the current work was submitted, the DuckieTown environment and Highway-Env [29] are not compatible with each other.

The resolution of traffic deadlocks solely in simulations should only be representative of what the behaviour in real life should really be like. It is important for the emergent policies to be accurately reproduced in reality, otherwise they remain toy examples.

At the current moment in time, Highway-Env, DuckieTown Gym [105] and possibly SMARTS [98] are the only representative simulators which can accurately emulate the behaviour needed for deadlock resolution (i.e. backwards movement of vehicles in traffic mobility scenarios). Neither of these environments, at this moment in time, have been improved yet with Sim2Real techniques [116] to permit the robustness of the emergent policies in real life scenarios.

Therefore, steps towards the inclusion of Sim2Real features in at least one of the aforementioned environments, the design of a novel simulator with these use cases in mind, or the extension of this work to environments like CARLA [117], would be a beneficial next step towards reaching safe behaviour in congested traffic scenarios.

6.4 Communication, Fairness & Scalability

In the pursuit of reaching a safe solution to the resolution of deadlocks in traffic, a set of measurement considerations were overlooked.

One of the first would be the ensurance of fairness in the resolution of the deadlock, in the case where the flow of incoming vehicles is longer than one car. In these circumstances, it is possible that the first lane to be served will continue emptying until it runs out of vehicles at deadlock resolution time, while the rest of the lanes suffer from starvation. In such a scenario, a look into how scheduling theory can be applied to traffic optimisation problems [118] could lead to a way in which the lanes can be resolved from the deadlock with fairness guarantees in mind.

On another note, in the work undertaken in this thesis, there has been the assumptions that the agents stuck in the deadlock were able to communicate with one another with a high degree of success and accuracy. This could not always be the case, due to the potential of communication packages to be lost or altered in transit. Literature into traffic coordination and V2V already considers the impact of communicating intentions in scenarios requiring high degrees of cooperation [6] in terms of both behaviour improvement and broadband consumption. In a deadlocked scenario and other high risk scenarios exhibiting high levels of congestions, there can be an increased importance in finding the balance between the performance gained through communication and the amount of communication that is required to reach it, without overflowing the broadband. Miscoordination in these environments could easily lead to accidents.

Furthermore, the scalability of the deadlock resolution problem has been under-evaluated in this current work. The MADDPG has been extended to use attention mechanisms, for better estimation of the value of the given environment in scenarios with fluctuating numbers of agents. Unfortunately, after training, no solution was able to resolve the deadlock in scenarios with 8 or more agents. The method suggested to solve the scalability situation would be transfer learning [119], with at least two approaches. The first approach would be to train the same agent on a smaller amount of agents, and gradually increase

this amount to cover more agents, while at the same time being sure to randomise the positioning of the given agents, for robustness under fluctuating throughput variations. The second approach would be to pre-train a traffic agent for basic coordination in general traffic situations, before asking it to solve potential deadlocks. This way, the agent learns the incentive of moving forwards when it can, and stopping or moving backwards when it observes an agent in front of it doing the same. It would be interesting to discover which approach behaves the best in this situation.

6.5 Human in the Loop

The assumption of having an entirely connected autonomously-driven civil transport system is both practically and ethically difficult to achieve. Therefore, agents who can be deployed in practice need to be able to cooperate not only with connected autonomous individuals, but with non-connected autonomous individuals and human drivers alike.

A recent contribution in this field is SymCoDrive [101], a Multi-Agent Reinforcement Learning approach. The cooperation is achieved through the creation of a specialised reward function, which includes a sympathetic incentive, allowing multiple agents to predict the behaviour of human drivers and provide cooperation based their apparent intentions in general driving scenarios. A limitation of this design could be the hand-crafted reward function, which might not be always the most optimal measurement of the throughput, depending on the traffic scenario, but it is still an important development to consider and expand upon for future work.

6.6 General Cooperation in Traffic

A noteworthy observation and limitation of both the related work on [MARL](#) in traffic simulators, and of this current work, is that the agents being trained are usually validated on the same type of environment or scenario, and there is little to no work done to train more general behaviour traffic models, with finer levels of control.

An approach in the future could be to train the same agents on multiple of these scenarios at the same time, such that these agents become generally good at cooperating in a multitude of situations. This will lead to agents which are more robust to scenario changes, even if they meet a situation which was never encountered during training. On the other hand, this approach could come at the detriment of a level of performance, which an agent fine tuned on a particular scenario would be able to outperform.

An alternative could be then to train the agents in a less end-to-end manner, and instead choose a hierarchical architecture approach. The agents could make use of two components: the first component identifies and classifies the scenario the agents are about to enter, and the second one, given the classification, chooses a particular trained policy to be undertaken. This approach can provide better guarantees of optimal performance in scenarios which the agents are prepared to tackle, but it can also be the case that the agent becomes overreliant on the classification aspect of the problem. As in, the misclassification, especially in cooperative scenarios, could lead to a lower degree of cooperation between the agents, since, if one can assume not all agents misclassify the scenario, some agents could express unexpected behaviour to other agents, leading to unpredictability in the environment.

This problem could be solved through initialising a consensus between the cooperative agents, which ensures that misclassifications are either circumvented, or that all agents are at least performing expected behaviour in relation to each other. The performance and modelling of V2X communication with the use of consensus in mind are elements already under development in the literature [120].

6.7 Validation of More MARL Architectures

A final suggestion would be to test further algorithms and architectures on the given problem, like Twin Delayed DDPG [45] and Soft Actor Critic [44], for both single and multi-agent variations. The purpose is to discover whether there is any particular framework which suits the Rebound from Failure subset of problems the best, and make comments on why that would be the case.

Chapter 7

Conclusion

To conclude, this thesis studied the resolution of deadlocks in traffic intersections through the use of reinforcement learning techniques. Through the development of this project, a number of contributions were made on two open source projects, namely on the *Highway-Env* traffic simulator, where two novel variations of the intersection environment being introduced, designed for the purpose of pursuing the training and validation of further algorithms to tackle the problem of resolving deadlocks, and on the *Rl-Agents* reinforcement learning library, with the implementation of a [MADDPG](#) algorithm framework, with variations that include self- and social-attention mechanisms.

A novel formalisation of the definition of a deadlock was proposed in this thesis, with modifications which make the potential occurrence of a deadlock detectable in continuous multi-agent scenarios without the need of several layers of abstraction to discretise the state space. On top of those, certain relaxations of the definition have been proposed, to make the concept of deadlocks more practical in open-space environments. To observe the notions of interest that can be captured, the formalisation has been discussed in the context of the deadlocked intersection environment tackled by this thesis.

Through the pursuit of the aforementioned goals, several observations were made regarding the state of the art in safety targets for reinforcement learning, highlighted by the limitations of current multi-agent traffic management. The majority of work in the literature focuses on robustness through prevention or tolerance to failure, which could lead to undertraining on unexpected dangerous scenarios. The case was made that, by shifting the focus more on the *rebound*, or recovery, rather than prevention of dangerous scenarios, the resulting policies will be safer.

The results of this study show that the resolution of deadlocks in multi-agent traffic scenarios is a challenging problem to optimise on, but one which, once solved, should prove autonomously driven agents can exhibit a high degree of mobility precision and cooperation. Future work proposed should focus on the preparation of a baseline multi-agent traffic coordination policy a priori, then to fine tune the agents for the deadlocked environment. The next important goal should be the improvement of the scalability of the policy highlighted to more autonomous agents, and more parallel lanes.

Glossary

AGV Autonomous Guided Vehicle.

AV Autonomous Vehicle.

CNN Convolutional Neural Network.

CTDE Centralised Training, Decentralised Execution.

CV Computer Vision.

DDPG Deep Deterministic Policy Gradient.

DL Deep Learning.

DQN Deep Q Network.

DRL Deep Reinforcement Learning.

FTCC Fault-Tolerant Cooperative Control.

HER Hindsight Experience Replay.

IDDPG Independent Deep Deterministic Policy Gradient.

MAAC Multi-Actor Attention Critic.

MADDPG Multi-Agent Deep Deterministic Policy Gradient.

MAPPO Multi-Agent Proximal Policy Optimisation.

MARL Multi-Agent Reinforcement Learning.

MAS Multi-Agent System.

MDP Markov Decision Process.

MLP Multi-Layered Perceptron.

NLP Natural Language Processing.

NN Neural Network.

PID Proportional-Integral-Derivative (controller).

POMDP Partially Observable Markovian Decision Process.

R-MADDPG Recurrent Multi-Agent Deep Deterministic Policy Gradient.

RL Reinforcement Learning.

RNN Recurrent Neural Network.

SotA State-of-the-Art.

V2V Vehicle-to-Vehicle.

V2X Vehicle-to-Everything.

VDN Value Decomposition Network.

Appendix A

Algorithms in Background

A.1 Deep Q-Learning

Algorithm 1 Deep Q-learning with Experience Replay

Initialise replay memory $\mathcal{D}$ to capacity N
for $episode = 1, M$ **do**
 Initialise state s_0
 for $t = 1, T$ **do**
 With probability ϵ select random action a_t
 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
 Execute action a_t in simulator and observe reward r_t and state s_{t+1}
 Store transition $\langle s_t, a_t, r_t, s_{t+1} \rangle$ in $\mathcal{D}$
 Sample random minibatch of transitions $\langle s_j, a_j, r_j, s_{j+1} \rangle$ from $\mathcal{D}$
 Set $y_j = \begin{cases} r_j & \text{for terminal } s_{j+1} \\ r_j + \gamma \max_{a'} Q^\pi(s_{j+1}, a'; \theta) & \text{for non-terminal } s_{j+1} \end{cases}$
 Perform a gradient descent step on $(y_j - Q^\pi(s_j, a_j; \theta))^2$
 end for
end for

A.2 Deep Deterministic Policy Gradient

Algorithm 2 Deep DPG with Experience Replay and Target Networks

Randomly initialise critic network $Q(s, a; \theta^Q)$ and actor $\pi(s; \theta^\pi)$ with weights θ^Q, θ^π .

Initialise target networks Q' and π' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\pi'} \leftarrow \theta^\pi$

Initialise replay buffer $\mathcal{D}$

for $episode = 1, M$ **do**

 Initialise a random process $\mathcal{N}$ for action exploration

 Initialise initial observation state s_0

for $t = 0, T$ **do**

 Select action $a_t = \pi(s_t; \theta^\pi) + \mathcal{N}_t$, w.r.t. current policy and exploration noise

 Execute action a_t and observe reward r_t and new state s_{t+1}

 Store transition $\langle s_t, a_t, r_t, s_{t+1} \rangle$ in $\mathcal{D}$

 Sample random minibatch of N transitions $\langle s_j, a_j, r_j, s_{j+1} \rangle$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_j & \text{for terminal } s_{j+1} \\ r_j + \gamma Q'(s_{j+1}, \pi'(s_{j+1}; \theta^{\pi'}); \theta^{Q'}) & \text{for non-terminal } s_{j+1} \end{cases}$

 Update critic by minimising loss $\mathcal{L} = \frac{1}{N} \sum_j (y_j - Q(s_j, a_j; \theta^Q))^2$

 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\pi} J \approx \frac{1}{N} \sum_j \nabla_a Q(s, a; \theta^Q)|_{s=s_j, a=\pi(s_j; \theta^\pi)} \nabla_{\theta^\pi} \pi(s; \theta^\pi)|_{s=s_j}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\pi'} \leftarrow \tau \theta^\pi + (1 - \tau) \theta^{\pi'}$$

end for
end for

A.3 Multi-Agent Deep Deterministic Policy Gradient (Actor-Critic)

Algorithm 3 Multi-Agent DDPG for N agents

```

for  $i = 1, N$  do
    Randomly initialise critic networks  $Q_i(\mathbf{x}, a; \theta_i^Q)$  and actor  $\pi_i(o; \theta_i^\pi)$ 
    Initialise target networks  $Q'_i$  and  $\pi'_i$  with weights  $\theta_i^{Q'} \leftarrow \theta_i^Q, \theta_i^{\pi'} \leftarrow \theta_i^\pi$ 
end for
Initialise replay buffer  $\mathcal{D}$ 
Initialise a random process  $\mathcal{N}$  for action exploration
for  $episode = 1, M$  do
    Receive initial state  $\mathbf{x}$ 
    for  $t = 1, T$  do
        for each agent  $i$ , select action  $a_i = \pi(o_i; \theta_i^\pi) + \mathcal{N}_t$ , w.r.t. current policy and exploration noise
        Execute actions  $a = (a_1, \dots, a_N)$  and observe reward  $r$  and new state  $\mathbf{x}'$ 
        Store transition  $\langle \mathbf{x}, a, r, \mathbf{x}' \rangle$  in  $\mathcal{D}$ 
        for agent  $i = 1, N$  do
            Sample random minibatch of  $S$  samples  $\langle \mathbf{x}^j, a^j, r^j, \mathbf{x}'^j \rangle$  from  $\mathcal{D}$ 
            Set  $y^j = r_i^j + \gamma Q'_i(\mathbf{x}'^j, a'_1, \dots, a'_N; \theta_i^{Q'})|_{a'_k = \pi'_k(o_k; \theta_k^\pi)}$ 
            Update critic by minimising loss  $\mathcal{L}(\theta_i^Q) = \frac{1}{S} \sum_j (y^j - Q_i(\mathbf{x}^j, a_1^j, \dots, a_N^j; \theta_i^Q))^2$ 
            Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta_i^\pi} J(\pi_i) \approx \frac{1}{S} \sum_j \nabla_{\theta_i^\pi} \pi_i(o_i^j; \theta_i^\pi) \nabla_{a_i} Q_i(\mathbf{x}^j, a_1^j, \dots, a_i, \dots, a_N^j; \theta_i^Q)|_{a_i = \pi_i(o_i; \theta_i^\pi)}$$

end for
            Update the target networks for each actor and critic  $i$ :

$$\theta_i^{\pi'} \leftarrow \tau \theta_i^\pi + (1 - \tau) \theta_i^{\pi'}$$


$$\theta_i^{Q'} \leftarrow \tau \theta_i^Q + (1 - \tau) \theta_i^{Q'}$$

end for
        end for
    end for

```

A.4 Multi-Agent Attention Critic

Algorithm 4 Multi-Agent Attention Critic for N agents

```

for  $i = 1, N$  do
  Randomly initialise critic networks  $Q_i(o, a; \theta_i^Q)$  and actor  $\pi_i(o_i; \theta_i^\pi)$ 
  Initialise target networks  $Q'_i$  and  $\pi'_i$  with weights  $\theta_i^{Q'} \leftarrow \theta_i^Q, \theta_i^{\pi'} \leftarrow \theta_i^\pi$ 
end for
Initialise replay buffer  $\mathcal{D}$ 
for  $episode = 1, M$  do
  Receive initial state  $\mathbf{x}$ 
  for  $t = 1, T$  do
    for each agent  $i$ , select action  $a_i \sim \pi(\cdot | o_i; \theta_i^\pi)$ , w.r.t. current policy
    Execute actions  $a = (a_1, \dots, a_N)$  and observe reward  $r$  and new observations  $\mathbf{o}'$ 
    Store transition  $\langle \mathbf{o}, a, r, \mathbf{o}' \rangle$  in  $\mathcal{D}$ 
    for agent  $i = 1, N$  do
      Sample random minibatch of  $S$  samples  $\langle \mathbf{o}^j, a^j, r^j, \mathbf{o}'^j \rangle$  from  $\mathcal{D}$ 
      Set  $y^j = r_i^j + \gamma \sum_{a' \in A} \pi'(a' | \mathbf{o}^j; \theta^{\pi'}) [Q'_i(\mathbf{o}^j, a'; \theta_i^{Q'}) - \alpha \log(\pi'_i(a' | \sigma_i^j; \theta_i^{\pi'}))]$ 
      Update critic by minimising loss  $\mathcal{L}(\theta_i^Q) = \frac{1}{S} \sum_j (y^j - Q_i(\mathbf{o}^j, a^j; \theta_i^Q))^2$ 
      Compute advantage for each agent  $A_i^j(\mathbf{o}^j, a^j) = Q_i(\mathbf{o}^j, a^j; \theta_i^Q) - b^j(\mathbf{o}^j, a_{\setminus i}^j)$ ,
      with baseline  $b^j(\mathbf{o}^j, a_{\setminus i}^j) = \sum_{a'_i \in A_i} \pi_i(a'_i | \sigma_i^j; \theta_i^\pi) Q_i(\mathbf{o}^j, (a'_i, a_{\setminus i}^j); \theta_i^Q)$ 
      Update the actor policy using the sampled policy gradient:
      
$$\nabla_{\theta_i^\pi} J(\pi_i) \approx \frac{1}{S} \sum_j \left[ \nabla_{\theta_i^\pi} \log(\pi_i(a_i | \sigma_i^j; \theta_i^\pi)) (-\alpha \log(\pi_i(a_i | \sigma_i^j; \theta_i^\pi)) + A_i(\mathbf{o}^j, a^j)) \right] \Big|_{a_i \sim \pi_i(a_i | \sigma_i^j; \theta_i^\pi)}$$

    end for
    Update the target networks for each actor and critic  $i$ :
    
$$\theta_i^{\pi'} \leftarrow \tau \theta_i^\pi + (1 - \tau) \theta_i^{\pi'}$$

    
$$\theta_i^{Q'} \leftarrow \tau \theta_i^Q + (1 - \tau) \theta_i^{Q'}$$

  end for
end for

```

Appendix B

Hyperparameters

B.1 Reward

B.1.1 Equation

$$r(t) = \frac{1}{N} \sum_{i \in N} \left(r_c(t) \text{ or } r_g(t) \text{ or } (v_i(t) - b_v) \right)$$

B.1.2 Hyperparameter table

Hyperparameter	Value	Description
$r_c(t)$	-100	penalty on crashed vehicles at time t or earlier
$r_g(t)$	30	reward on goal reached at time t or earlier
b_v	3.5	velocity reward bias, penalising stationary or backwards-moving vehicles

Table B.1: Table of Hyperparameters for reward engineering

B.2 Training Networks

B.2.1 Hyperparameter Table

Hyperparameter	Value		Description
	DQN	MADDPG	
γ	0.95		discount factor for value estimation update
batch size	512	256	number of frames over which network updates are made
memory capacity	50000	25000	max amount of frames stored at once in replay buffer
target update	1/512 episodes	soft $\tau = 0.01$	target network update method and rate
initial ϵ	1.0		starting value of greedy exploration coefficient
final ϵ	0.05		final value of greedy exploration coefficient
ϵ decay	linear 150000 frames	linear 50000 frames	method and rate of decay for exploration coefficient

Table B.2: Table of Hyperparameters for DQN and MADDPG variations, respectively

Bibliography

- [1] Grigorescu S, Trasnea B, Cocias T, Macesanu G. A survey of deep learning techniques for autonomous driving. *Journal of Field Robotics*. 2020;37(3):362-86.
- [2] Zhu H, Yuen KV, Mihaylova L, Leung H. Overview of environment perception for intelligent vehicles. *IEEE Transactions on Intelligent Transportation Systems*. 2017;18(10):2584-601.
- [3] Wang J, Shao Y, Ge Y, Yu R. A survey of vehicle to everything (V2X) testing. *Sensors*. 2019;19(2):334.
- [4] O’Kane S. How Tesla and Waymo are tackling a major problem for self-driving cars: Data. *Transportation*. 2018.
- [5] Friedrich B. The effect of autonomous vehicles on traffic. In: *Autonomous Driving*. Springer; 2016. p. 317-34.
- [6] Kim W, Park J, Sung Y. Communication in multi-agent reinforcement learning: Intention sharing. In: *International Conference on Learning Representations*; 2020.
- [7] Zhang R, Spieser K, Frazzoli E, Pavone M. Models, algorithms, and evaluation for autonomous mobility-on-demand systems. In: *2015 American Control Conference (ACC)*. IEEE; 2015. p. 2573-87.
- [8] Litman T. Autonomous vehicle implementation predictions: Implications for transport planning. 2020.
- [9] Fagnant DJ, Kockelman K. Preparing a nation for autonomous vehicles: opportunities, barriers and policy recommendations. *Transportation Research Part A: Policy and Practice*. 2015;77:167-81.
- [10] Zhong Z, Nejad M, Lee EE. Autonomous and Semiautonomous Intersection Management: A Survey. *IEEE Intelligent Transportation Systems Magazine*. 2020;13(2):53-70.
- [11] Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, et al. Human-level control through deep reinforcement learning. *nature*. 2015;518(7540):529-33.
- [12] Silver D, Hubert T, Schrittwieser J, Antonoglou I, Lai M, Guez A, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*. 2018;362(6419):1140-4.
- [13] Silver D, Huang A, Maddison CJ, Guez A, Sifre L, Van Den Driessche G, et al. Mastering the game of Go with deep neural networks and tree search. *nature*. 2016;529(7587):484-9.

- [14] Berner C, Brockman G, Chan B, Cheung V, Dębiak P, Dennison C, et al. Dota 2 with large scale deep reinforcement learning. arXiv preprint arXiv:1912.06680. 2019.
- [15] Baker B, Kanitscheider I, Markov T, Wu Y, Powell G, McGrew B, et al. Emergent tool use from multi-agent autocurricula. arXiv preprint arXiv:1909.07528. 2019.
- [16] Dixit VV, Chand S, Nair DJ. Autonomous vehicles: disengagements, accidents and reaction times. PLoS one. 2016;11(12):e0168054.
- [17] Koopman P, Wagner M. Autonomous vehicle safety: An interdisciplinary challenge. IEEE Intelligent Transportation Systems Magazine. 2017;9(1):90-6.
- [18] Colson B, Marcotte P, Savard G. An overview of bilevel optimization. Annals of operations research. 2007;153(1):235-56.
- [19] Garcia J, Fernández F. A comprehensive survey on safe reinforcement learning. Journal of Machine Learning Research. 2015;16(1):1437-80.
- [20] Gu S, Yang L, Du Y, Chen G, Walter F, Wang J, et al. A Review of Safe Reinforcement Learning: Methods, Theory and Applications. arXiv preprint arXiv:2205.10330. 2022.
- [21] Petrović Đ, Mijailović R, Pešić D. Traffic accidents with autonomous vehicles: type of collisions, manoeuvres and errors of conventional vehicles' drivers. Transportation research procedia. 2020;45:161-8.
- [22] Al-Nima RRO, Han T, Al-Sumaidae SAM, Chen T, Woo WL. Robustness and performance of deep reinforcement learning. Applied Soft Computing. 2021;105:107295.
- [23] Brunke L, Greeff M, Hall AW, Yuan Z, Zhou S, Panerati J, et al. Safe learning in robotics: From learning-based control to safe reinforcement learning. arXiv preprint arXiv:2108.06266. 2021.
- [24] Goodfellow IJ, Shlens J, Szegedy C. Explaining and harnessing adversarial examples. arXiv preprint arXiv:1412.6572. 2014.
- [25] Atkeson CG, Babu BPW, Banerjee N, Berenson D, Bove CP, Cui X, et al. No falls, no resets: Reliable humanoid behavior in the DARPA robotics challenge. In: 2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids). IEEE; 2015. p. 623-30.
- [26] Arnett J. Reckless behavior in adolescence: A developmental perspective. Developmental review. 1992;12(4):339-73.
- [27] Karni E, Schmeidler D. Self-preservation as a foundation of rational behavior under risk. Journal of Economic Behavior & Organization. 1986;7(1):71-81.
- [28] De Ryck M, Versteyhe M, Debrouwere F. Automated guided vehicle systems, state-of-the-art control algorithms and techniques. Journal of Manufacturing Systems. 2020;54:152-73.
- [29] Leurent E. An Environment for Autonomous Driving Decision-Making. GitHub; 2018. <https://github.com/eleurent/highway-env>.
- [30] Brockman G, Cheung V, Pettersson L, Schneider J, Schulman J, Tang J, et al.. OpenAI Gym; 2016.

- [31] Leurent E. rl-agents: Implementations of Reinforcement Learning algorithms. GitHub; 2018. <https://github.com/eleurent/rl-agents>.
- [32] Ferber J, Weiss G. Multi-agent systems: an introduction to distributed artificial intelligence. vol. 1. Addison-Wesley Reading; 1999.
- [33] Vlassis N. A concise introduction to multiagent systems and distributed artificial intelligence. *Synthesis Lectures on Artificial Intelligence and Machine Learning*. 2007;1(1):1-71.
- [34] Netzer RH, Miller BP. What are race conditions? Some issues and formalizations. *ACM Letters on Programming Languages and Systems (LOPLAS)*. 1992;1(1):74-88.
- [35] Coffman EG, Elphick M, Shoshani A. System deadlocks. *ACM Computing Surveys (CSUR)*. 1971;3(2):67-78.
- [36] Singhal M. Deadlock detection in distributed systems. *Computer*. 1989;22(11):37-48.
- [37] Mnih V, Kavukcuoglu K, Silver D, Graves A, Antonoglou I, Wierstra D, et al. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:13125602*. 2013.
- [38] Lillicrap TP, Hunt JJ, Pritzel A, Heess N, Erez T, Tassa Y, et al. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:150902971*. 2015.
- [39] Bellman R. A Markovian decision process. *Journal of mathematics and mechanics*. 1957:679-84.
- [40] Sutton RS, Barto AG, et al. Introduction to reinforcement learning. 1998.
- [41] Watkins CJ, Dayan P. Q-learning. *Machine learning*. 1992;8(3):279-92.
- [42] Silver D, Lever G, Heess N, Degris T, Wierstra D, Riedmiller M. Deterministic policy gradient algorithms. In: *International conference on machine learning*. PMLR; 2014. p. 387-95.
- [43] Lin LJ. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*. 1992;8(3):293-321.
- [44] Haarnoja T, Zhou A, Hartikainen K, Tucker G, Ha S, Tan J, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:181205905*. 2018.
- [45] Fujimoto S, Hoof H, Meger D. Addressing function approximation error in actor-critic methods. In: *International conference on machine learning*. PMLR; 2018. p. 1587-96.
- [46] Schaul T, Quan J, Antonoglou I, Silver D. Prioritized experience replay. *arXiv preprint arXiv:151105952*. 2015.
- [47] Andrychowicz M, Wolski F, Ray A, Schneider J, Fong R, Welinder P, et al. Hindsight experience replay. *Advances in neural information processing systems*. 2017;30.
- [48] Akkaya I, Andrychowicz M, Chociej M, Litwin M, McGrew B, Petron A, et al. Solving rubik’s cube with a robot hand. *arXiv preprint arXiv:191007113*. 2019.
- [49] Tan M. Multi-agent reinforcement learning: Independent vs. cooperative agents. In: *Proceedings of the tenth international conference on machine learning*; 1993. p. 330-7.

- [50] Williams JD, Young S. Partially observable Markov decision processes for spoken dialog systems. *Computer Speech & Language*. 2007;21(2):393-422.
- [51] Spaan MT. Partially observable Markov decision processes. In: *Reinforcement Learning*. Springer; 2012. p. 387-414.
- [52] Oliehoek FA, Spaan MT, Vlassis N. Optimal and approximate Q-value functions for decentralized POMDPs. *Journal of Artificial Intelligence Research*. 2008;32:289-353.
- [53] Aşık O, Akın HL. Solving multi-agent decision problems modeled as dec-pomdp: A robot soccer case study. In: *Robot Soccer World Cup*. Springer; 2012. p. 130-40.
- [54] Oliehoek FA, Amato C. A concise introduction to decentralized POMDPs. Springer; 2016.
- [55] Buşoniu L, Babuška R, Schutter BD. Multi-agent reinforcement learning: An overview. *Innovations in multi-agent systems and applications-1*. 2010:183-221.
- [56] Lowe R, Wu YI, Tamar A, Harb J, Pieter Abbeel O, Mordatch I. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in neural information processing systems*. 2017;30.
- [57] Rashid T, Samvelyan M, Schroeder C, Farquhar G, Foerster J, Whiteson S. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning. In: *International Conference on Machine Learning*. PMLR; 2018. p. 4295-304.
- [58] Yu C, Velu A, Vinitisky E, Wang Y, Bayen A, Wu Y. The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games. *arXiv preprint arXiv:210301955*. 2021.
- [59] Sunehag P, Lever G, Gruslys A, Czarnecki WM, Zambaldi V, Jaderberg M, et al. Value-decomposition networks for cooperative multi-agent learning. *arXiv preprint arXiv:170605296*. 2017.
- [60] Foerster J, Farquhar G, Afouras T, Nardelli N, Whiteson S. Counterfactual multi-agent policy gradients. In: *Proceedings of the AAAI conference on artificial intelligence*. vol. 32; 2018. .
- [61] Sycara K, Pannu A, Williamson M, Zeng D, Decker K. Distributed intelligent agents. *IEEE expert*. 1996;11(6):36-46.
- [62] Iqbal S, Sha F. Actor-attention-critic for multi-agent reinforcement learning. In: *International Conference on Machine Learning*. PMLR; 2019. p. 2961-70.
- [63] Hochreiter S, Schmidhuber J. Long short-term memory. *Neural computation*. 1997;9(8):1735-80.
- [64] Cho K, Van Merriënboer B, Bahdanau D, Bengio Y. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:14091259*. 2014.
- [65] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. *Advances in neural information processing systems*. 2017;30.
- [66] Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:14090473*. 2014.
- [67] Devlin J, Chang MW, Lee K, Toutanova K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:181004805*. 2018.

- [68] Liu Y, Ott M, Goyal N, Du J, Joshi M, Chen D, et al. Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692. 2019.
- [69] Lan Z, Chen M, Goodman S, Gimpel K, Sharma P, Soricut R. Albert: A lite bert for self-supervised learning of language representations. arXiv preprint arXiv:1909.11942. 2019.
- [70] Mnih V, Heess N, Graves A, et al. Recurrent models of visual attention. *Advances in neural information processing systems*. 2014;27.
- [71] Xu K, Ba J, Kiros R, Cho K, Courville A, Salakhudinov R, et al. Show, attend and tell: Neural image caption generation with visual attention. In: *International conference on machine learning*. PMLR; 2015. p. 2048-57.
- [72] Han K, Wang Y, Chen H, Chen X, Guo J, Liu Z, et al. A survey on vision transformer. *IEEE transactions on pattern analysis and machine intelligence*. 2022.
- [73] Chen F, Zhang D, Han M, Chen X, Shi J, Xu S, et al. Vlp: A survey on vision-language pre-training. arXiv preprint arXiv:2202.09061. 2022.
- [74] Oh J, Chockalingam V, Lee H, et al. Control of memory, active perception, and action in minecraft. In: *International Conference on Machine Learning*. PMLR; 2016. p. 2790-9.
- [75] Leurent E, Mercat J. Social attention for autonomous decision-making in dense traffic. arXiv preprint arXiv:1911.12250. 2019.
- [76] Yang JM, Kim JH. Fault-tolerant locomotion of the hexapod robot. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*. 1998;28(1):109-16.
- [77] Yang JM. Fault-tolerant gaits of quadruped robots for locked joint failures. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*. 2002;32(4):507-16.
- [78] Koos S, Cully A, Mouret JB. Fast damage recovery in robotics with the t-resilience algorithm. *The International Journal of Robotics Research*. 2013;32(14):1700-23.
- [79] Cully A, Clune J, Tarapore D, Mouret JB. Robots that can adapt like animals. *Nature*. 2015;521(7553):503-7.
- [80] Li B, Zhang Y. Fault-tolerant cooperative motion planning of connected and automated vehicles at a signal-free and lane-free intersection. *IFAC-PapersOnLine*. 2018;51(24):60-7.
- [81] Shoshani A, Coffman E. Prevention, detection, and recovery from system deadlocks. In: *Proc. 4th annual Princeton Conf. on Information Sciences and System*; 1970. .
- [82] Wang S, Wan J, Zhang D, Li D, Zhang C. Towards smart factory for industry 4.0: a self-organized multi-agent system with big data based feedback and coordination. *Computer networks*. 2016;101:158-68.
- [83] Duhaut D, Carrillo E, Saint-Aimé S. Avoiding deadlock in multi-agent systems. In: *2007 IEEE International Conference on Systems, Man and Cybernetics*. IEEE; 2007. p. 1642-7.
- [84] Kujirai T, Yokota T. Breaking Deadlocks in Multi-agent Reinforcement Learning with Sparse Interaction. In: *Pacific Rim International Conference on Artificial Intelligence*. Springer; 2019. p. 746-59.

- [85] Alshiekh M, Bloem R, Ehlers R, Könighofer B, Niekum S, Topcu U. Safe reinforcement learning via shielding. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 32; 2018. .
- [86] Wurman PR, D’Andrea R, Mountz M. Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI magazine*. 2008;29(1):9-9.
- [87] Angeloudis P, Bell MG. An uncertainty-aware AGV assignment algorithm for automated container terminals. *Transportation Research Part E: Logistics and Transportation Review*. 2010;46(3):354-66.
- [88] Seiler KM, Palmer AW, Hill AJ. Flow-achieving online planning and dispatching for continuous transportation with autonomous vehicles. *IEEE Transactions on Automation Science and Engineering*. 2020.
- [89] Daganzo CF, Geroliminis N. An analytical approximation for the macroscopic fundamental diagram of urban traffic. *Transportation Research Part B: Methodological*. 2008;42(9):771-81.
- [90] Daganzo CF. Urban gridlock: Macroscopic modeling and mitigation approaches. *Transportation Research Part B: Methodological*. 2007;41(1):49-62.
- [91] Kerner BS. Physics of traffic gridlock in a city. *Physical Review E*. 2011;84(4):045102.
- [92] Saprykin A, Chokani N, Abhari RS. Gridlock resolution in a GPU-accelerated traffic queue model. *Procedia Computer Science*. 2020;170:681-7.
- [93] Wang N, Wang X, Palacharla P, Ikeuchi T. Cooperative autonomous driving for traffic congestion avoidance through vehicle-to-vehicle communications. In: 2017 IEEE Vehicular Networking Conference (VNC). IEEE; 2017. p. 327-30.
- [94] Janai J, Güney F, Behl A, Geiger A, et al. Computer vision for autonomous vehicles: Problems, datasets and state of the art. *Foundations and Trends® in Computer Graphics and Vision*. 2020;12(1–3):1-308.
- [95] Anavatti SG, Francis SL, Garratt M. Path-planning modules for Autonomous Vehicles: Current status and challenges. In: 2015 International Conference on Advanced Mechatronics, Intelligent Manufacture, and Industrial Automation (ICAMIMIA). IEEE; 2015. p. 205-14.
- [96] Henriksson J, Borg M, Englund C. Automotive safety and machine learning: Initial results from a study on how to adapt the ISO 26262 safety standard. In: 2018 IEEE/ACM 1st International Workshop on Software Engineering for AI in Autonomous Systems (SEFAIAS). IEEE; 2018. p. 47-9.
- [97] Krajzewicz D, Erdmann J, Behrisch M, Bieker L. Recent development and applications of SUMO-Simulation of Urban MObility. *International journal on advances in systems and measurements*. 2012;5(3&4).
- [98] Zhou M, Luo J, Vilella J, Yang Y, Rusu D, Miao J, et al.. SMARTS: Scalable Multi-Agent Reinforcement Learning Training School for Autonomous Driving; 2020. Available from: <https://arxiv.org/abs/2010.09776>.
- [99] Paull L, Tani J, Ahn H, Alonso-Mora J, Carlone L, Cap M, et al. Duckietown: an open, inexpensive and flexible platform for autonomy education and research. In: 2017 IEEE International Conference on Robotics and Automation (ICRA); 2017. p. 1497-504.

- [100] Candela E, Parada L, Marques L, Georgescu TA, Demiris Y, Angeloudis P. Transferring Multi-Agent Reinforcement Learning Policies for Autonomous Driving using Sim-to-Real. arXiv preprint arXiv:220311653. 2022.
- [101] Toghi B, Valiente R, Sadigh D, Pedarsani R, Fallah YP. Cooperative autonomous vehicles that sympathize with human drivers. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE; 2021. p. 4517-24.
- [102] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal policy optimization algorithms. arXiv preprint arXiv:170706347. 2017.
- [103] Wang RE, Everett M, How JP. R-MADDPG for partially observable environments and limited communication. arXiv preprint arXiv:200206684. 2020.
- [104] Li B, Liang S, Gan Z, Chen D, Gao P. Research on multi-UAV task decision-making based on improved MADDPG algorithm and transfer learning. International Journal of Bio-Inspired Computation. 2021;18(2):82-91.
- [105] Chevalier-Boisvert M, Golemo F, Cao Y, Mehta B, Paull L. Duckietown Environments for OpenAI Gym. GitHub; 2018. <https://github.com/duckietown/gym-duckietown>.
- [106] Kheterpal N, Parvate K, Wu C, Kreidieh A, Vinitisky E, Bayen A. Flow: Deep reinforcement learning for control in sumo. EPiC Series in Engineering. 2018;2:134-51.
- [107] Raffin A, Hill A, Gleave A, Kanervisto A, Ernestus M, Dormann N. Stable-Baselines3: Reliable Reinforcement Learning Implementations. Journal of Machine Learning Research. 2021;22(268):1-8. Available from: <http://jmlr.org/papers/v22/20-1364.html>.
- [108] Liang E, Liaw R, Nishihara R, Moritz P, Fox R, Goldberg K, et al. RLlib: Abstractions for distributed reinforcement learning. In: International Conference on Machine Learning. PMLR; 2018. p. 3053-62.
- [109] Abadi M, Agarwal A, Barham P, Brevdo E, Chen Z, Citro C, et al.. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems; 2015. Software available from tensorflow.org. Available from: <https://www.tensorflow.org/>.
- [110] Yang H, Han QL, Ge X, Ding L, Xu Y, Jiang B, et al. Fault-tolerant cooperative control of multiagent systems: A survey of trends and methodologies. IEEE Transactions on Industrial Informatics. 2019;16(1):4-17.
- [111] Fulton N, Platzer A. Safe reinforcement learning via formal methods: Toward safe control through proof and learning. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 32; 2018. .
- [112] Sidorov D. Integral dynamical models: singularities, signals and control. vol. 87. World Scientific; 2014.
- [113] Thomson JJ. The Trolley Problem/Das Trolley-Problem (Englisch/Deutsch): Reclam Great Papers Philosophie. Reclam Verlag; 2020.
- [114] Hevelke A, Nida-Rümelin J. Responsibility for crashes of autonomous vehicles: An ethical analysis. Science and engineering ethics. 2015;21(3):619-30.

- [115] Orel FD, Kara A. Supermarket self-checkout service quality, customer satisfaction, and loyalty: Empirical evidence from an emerging market. *Journal of Retailing and Consumer services*. 2014;21(2):118-29.
- [116] Zhao W, Queralta JP, Westerlund T. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In: *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE; 2020. p. 737-44.
- [117] Dosovitskiy A, Ros G, Codevilla F, Lopez A, Koltun V. CARLA: An open urban driving simulator. In: *Conference on robot learning*. PMLR; 2017. p. 1-16.
- [118] Giridhar A, Kumar P. Scheduling automated traffic on a network of roads. *IEEE Transactions on Vehicular Technology*. 2006;55(5):1467-74.
- [119] Pan SJ, Yang Q. A survey on transfer learning. *IEEE Transactions on knowledge and data engineering*. 2009;22(10):1345-59.
- [120] Wang Z, Wu G, Barth M. Distributed consensus-based cooperative highway on-ramp merging using V2X communications. *SAE Technical Paper*; 2018.